System Engineering

with Minimal Magic

Pum Walters

PUBLISHED BY PUBLISHER

BOOK-WEBSITE.COM

First printing, December 2021

Contents

References

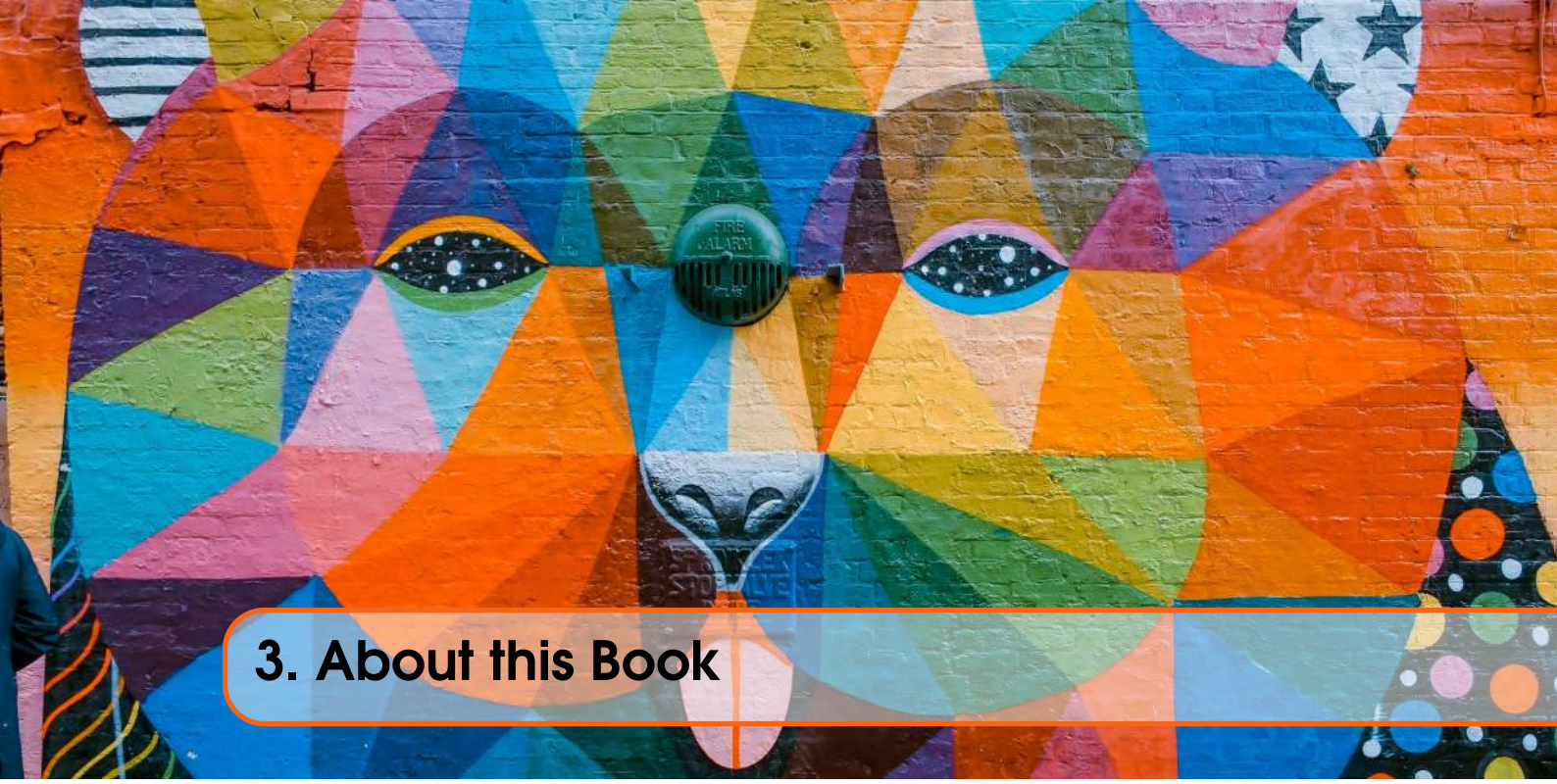
1. Minimal Magic

I once gave a freshman course on system engineering. The magic of system engineering is that once you understand a transistor, you can mostly understand an entire CPU. Later, I started writing a book on term rewriting with a similar foundation: *Ex Principiis Omnia*. This gave me the idea to start a blog with work and musings on that underlying theme.

2. Pum Walters

I am an independent researcher in Computer Science (Software Engineering, Cybersecurity, Enterprise Architecture, and Technical Computing). I have written various articles on Computer Science & Education and many parts of *as-yet* unpublished books and stories (both science and fiction).

Over the years, I've had many ideas for papers or books, and I only just now realized they often share a common theme, which is the reason to create this blog and which is expressed in the motto ***Ex Principiis Omnia***: everything follows from the beginning with **minimal magic**.



3. About this Book

This book is a direct (automatic) rendition of the blog Minimal Magic ( ) with *only* changes that affect the graphic design and leave further content as-is. Currently, internal links in the book refer to the blog.

All photo's are taken (with permission) from the wonderful site [Pexels](https://www.pexels.com) or elsewhere when explicitly mentioned.

All other images are by Pum Walters.

Part One

4. Some Computing History

4.1 Oldest computers

In many introductions to computing, the **Antikythera mechanism** is mentioned, which is a 60-gear mechanical astrological calculator made about 2000 years ago.

Arguably, the ***1st Stonehenge*** is a much older example.

Figure 4.1: Stonehenge

Who doesn't know the huge Neolithic standing stones and lintels? But in fact, the stones are the 3rd Stonehenge. Around the stones is a circle of pits which predate the stones by a millennium. One theory suggests that colored pebbles were put in the pits and advanced by priests each day from one pit to the next.

Using the location of the pebbles, astronomical predictions such as eclipses and equinoxes can be made about the positions of the Sun, Moon, stars and planets.

That would mean the system uses data (the position of pebbles) and a hard-wired model (the placing of the holes) to model and make predictions of real world events.

Since the 1st Stonehenge is about 5000 years old, this would mean 'computing' is as old as writing!

4.2 Mechanical computing

- An ***abacus*** is a frame containing beads capable of representing large integers and performing (by hand) operators including addition, subtraction, multiplication, division, and taking square and cube roots.
- A ***slide rule*** represents real numbers by distances and can perform multiplication, division and other mathematical calculations.
- ~ 1800:

  - the first ‘mass produced’ mechanical calculator
  - the first looms in which patterns were coded in punched cards
  - the (incomplete) design of Charles Babbage’s Analytical Engine, which includes 16K memory, an ALU, control flow and even conditional branching. Note that this engine has never been built.

4.3 Electric Computers

Highlights:

- The first programmable electromechanical (relay-) computers: Zuse
- Mathematics and computing are closely related (Turing, Gödel)
- Von Neumann model: program is data
- 40’s: vacuum tubes
- 50’s: transistors
- 60’s: integrated circuits
- 20’s: quantum computing

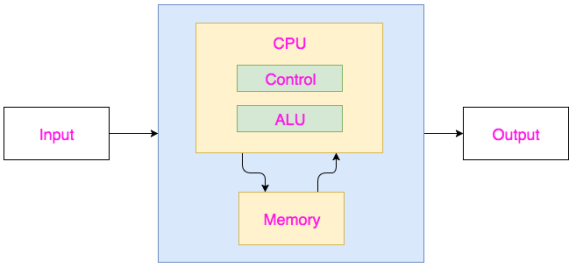


Figure 4.3: von Neumann Architecture

5. System Engineering - Introduction

A current phone has more sensors (15+) and substantially more computing power (8+ 64-bit cores) than NASA had in the 60s to land on the moon. It can video a year of your life, non-stop, easily. It weighs about 150 grams.

How is that possible? It is a cutting edge of System Engineering.

5.1 Engineering

Engineering is about

- finding solutions to problems and
- making use of opportunities

⊙**Example**

A candle-snuffer is a small pair of scissors with a tiny box attached, used to cut a candle's wick before it starts producing black smoke due to bad combustion. *Most are antique because modern candles don't need them.*

One thread in a modern wick is pulled taut when the candle is made, resulting in the wick curling when it is released from the wax. The curling wick sticks out of the flame, where it burns entirely, preventing it from becoming too long and resulting in incomplete combustion of wax. Pulling one wire taut is an engineering solution.

⊙**Example**

An inventor created 'bad glue' which did not work well on paper, and not at all on other materials. For six years, it was considered useless. *Today PostIt is a multi-billion product and is used around the world!*.

⊙**Example**

Analog computers use voltage to represent values. But voltages can change based upon temperature, humidity, earth magnetism. Imagine that your salary depends on the weather. Engineers solved this problem by *understanding that precise voltages change, but the absence or presence of voltage within bounds does not*.

Today a phone has > 1.000.000.000 transistors switching (determining the presence or absence of voltage) > 1.000.000.000 times per second.

5.2 Layers

In order to understand systems with thousands of millions of parts, humans need to apply structure. Just as the TCP/IP stack is used to understand computer networks, another layered model is used to understand systems.

Whereas the TCP/IP stack is (part of) an international (ISO) standard, the 6-level structured computer organization is not. **Why not?**

$\Longrightarrow$

Participants in a network must agree on a standard before they can communicate; an individual system could be structured in many different ways. Standardization only follows after commoditization in parts and services.

Nevertheless, most autonomous digital systems can be viewed in this layered model.

- Problem Oriented Language Level
  Most programs and languages we use day to day exist at this level. Programs such as browsers, word processors, editors, etcetera, and languages such as Java, JavaScript, HTML and SQL exist solely at this level.
- Assembly Language Level
  Languages such as Java are called 'high' because they deal with abstract concepts suitable for humans to work on. Other languages deal with simpler concepts (e.g. numbers) more suitable for processing directly by hardware.
- Operating System Machine Level
  The primary role of an OS is to manage all resources. The OS has drivers (specialized software to manage specific hardware), and it has many algorithms (programs) to allow processes access to these resources.
- Instruction Set Architecture Level
  How is it possible that the same software (such as a web browser) runs on different hardware (e.g. i3, i5, i9, AMD)? The hardware simulates one particular processor, which is called an Instruction Set Architecture (ISA).
- Microarchitecture Level
  It follows that a level must exist below ISA. This is called the microarchitecture.
- Digital Logic Level
  The lowest level consists of: transistors and other primitive electronic parts. Transistors are used to build larger components. For instance, two transistors are used to create one AND-gate: a component which computes the Boolean function AND. Hence the name **Digital Logic**.

6. Transistors & Gates

6.1 Semiconductors

A semiconductor is a material which conducts electricity, but not too well. Worse than conductors such as most metals, but much better than insulators such as most plastics.

There are two types of semiconductor:

- N-type, which has an abundance of loosely coupled electrons
- P-type, which has an abundance of 'holes' where electrons could bind

The P-type substrate prevents current from flowing from the source (left) to the drain (right). When voltage is applied to the gate, the positive charge in the dielectric layer induces a field in the P-type substrate which allows electrons to flow to the drain.

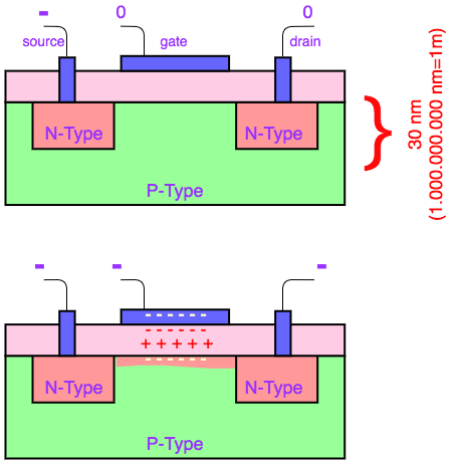


Figure 6.1: Transistors

Visit Veritasium's explanation of transistors on YouTube.

Simply put: no voltage on the gate means no voltage on the drain, whatever the voltage on the source; and a voltage on the gate means that the drain has a voltage precisely when the source has one. Simpler still: the gate determines if the drain copies the source or is zero.

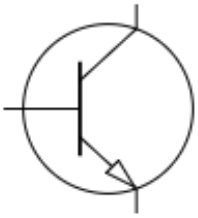


Figure 6.2: Transistor Symbol

There are many types of transistors based on these same principles. BJT-type transistors amplify current and are used in audio amplifiers. MOSFET-type transistors operate on voltage with very little current and are more suitable in computer switching. We have shown enhancement-mode N-channel MOSFETS, which is CMOS technology as used in current computers.

In this blog we are interested in (logical) system engineering rather than electronic engineering,

so we use simplified a design.

6.2 Digital Computers

As mentioned, in a digital computer the presence or absence of voltage (rather than the precise voltage) can be interpreted as binary data. A line with a (near) zero voltage represents 0; a line with a higher voltage represents 1. So, anything between 0 and, say, 0.8 volt is interpreted as 0, and anything from 2 to 5 volts as 1. Circuitry is designed such that voltages in the area between are avoided (and would lead to ambiguous results). Also, designers chose whether positive or negative voltages are used to represent non-zero values. Often -5v represents 1.

Based on this, transistors can be used to compute values. For instance, two (TTL-type) transistors in series compute the Boolean **AND** function: electricity can only flow if both transistors allow it.

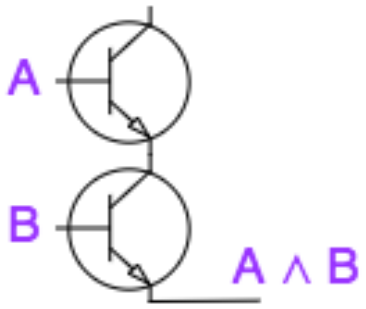


Figure 6.4: AND-Gate

6.3 Gates

A Logic **Gate** is a component consisting of a few transistors (with supporting elements such as diodes or resistors) computing some Boolean function.
Logic gates exist for all primitive Boolean operators, but

- depending upon the specific technology, one gate needs more transistors than another, and
- as we will learn, all Boolean operators can be computed using others. For instance, the NAND can compute *all other Boolean operators*

Because of this, different types of technology (TTL, NMOS, CMOS) favor specific gates which use less space or time.

*Note: confusingly, the part in a transistor that controls flow is called a **gate** and a component made of a few transistors is also called a **gate**.*

⊘

Gates form the unit of design for digital electronics (although larger modules and modularization techniques exist). For example, visit `https://hackaday.io/project/9795-nedonand-homebrew-computer` for a project in which an 8-bit processor is created out of 74F00-chips, each containing two NAND-gates.

Exercise:

- Create an inverter (that is a NOT operator) using only NAND gates.
- Create an OR gate using only NAND gates

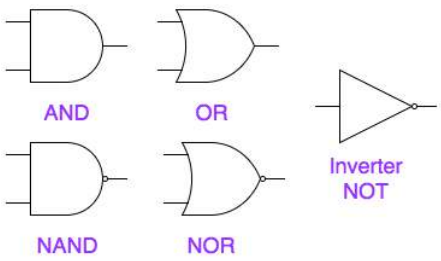


Figure 6.5: Different Gates

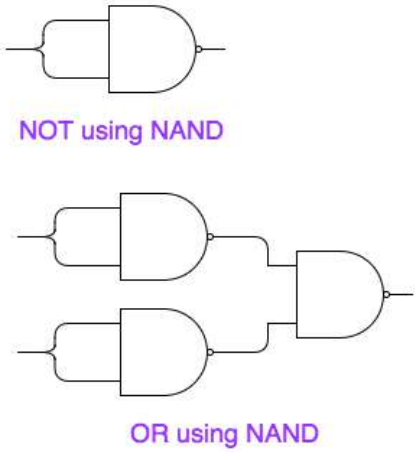


Figure 6.6: Inverter + OR using AND

Part Two

7. Boolean Arithmetic - Numbers

7.1 Booleans

There are two values: ***true*** and ***false***. Many operations, such as *comparisons*, on other data (e.g. numbers, dates) result in a boolean. For instance: *x>5* is *true* if *x* exceeds 5 and is *false* otherwise.

Truth values can be combined or computed using Boolean operators.

AND yields *true* only if both its arguments are *true*, and yields *false* otherwise (i.e. if one argument or both are *false*).

For example, for example, you can only join this park ride if your length is between 1.40 and 2.10, and your weight does not exceed 150Kg. `canJoin = (len > 1.40 AND len < 2.10) AND weight < 150`

Other operators include **OR**, **NAND**, **NOR** and **NOT**.

7.1.1 Boolean Arithmetic

It is sometimes convenient to represent Boolean values as numbers: 0 (*false*) and 1 (*true*). A single bit can be interpreted as an integer or as a Boolean.

When 0 and 1 are used to represent truth values, the **AND** operator is often written as * . Note that multiplication, when limited to 0 and 1, does indeed compute **AND**: *p* * *q* only equals 1 if both *p* and *q* are 1.

Similarly, **OR** is written as + and **NOT** is written as unary -. Note that Integer and Boolean + and (unary) - do not coincide with the integer operations: Boolean 1+1 is 1 and -1 is 0.

In many programming languages, AND and OR are often written as `&` and `|`.

*In programming, identifiers **true** and **false** are generally used to avoid confusion with integers, although in many languages integers can be used as 'truth values'.*

7.2 Truth Tables

Truth Tables are a technique to compute the value of Boolean expressions. In a truth table all possible values of all variables are listed together with the value of the expression (or its sub-expressions if the expression is complex).

The truth table of NOT and AND are:

NOT	
p	-p
0	1
1	0

AND		
p	q	p*q
0	0	0
1	0	0
0	1	0
1	1	1

Figure 7.1: Truth Tables

7.2.1 Complex Truth Tables

Using truth tables, the value of complex Boolean expressions can be computed even if their value isn't obvious immediately. For instance: what is the value of

```
(p or q ) and not (not p and not q)
```

Without the truth table it is not immediately clear that this expression is always *false*.

An important operator is XOR (exclusive or): *p xor q = (p or q) and not (p and q)*.

This operator is important because unlike AND and OR it does not lose information and is therefore useful in encryption: (p xor q) xor p = q. To the right is a common symbol for an XOR gate.

Consider the truth tables of `(not p and q) or (p and not q)` and `p xor q`. From the tables, we see that they are identical!

(p or q) and not (not p and not q)						
p	q	p+q	-p	-q	-p*-q	(p+q)*-(-p*-q)
0	0	0	1	1	1	0
1	0	0	0	1	0	0
0	1	0	1	0	0	0
1	1	1	0	0	0	0

Figure 7.2: Complex Truth Tables

XOR		
p	q	p⊕q
0	0	0
1	0	1
0	1	1
1	1	0

(not p and q) or (p and not q)						
p	q	-p	-q	-p*q	p*-q	(-p*q)+(p*-q)
0	0	1	1	0	0	0
1	0	0	1	0	1	1
0	1	1	0	1	0	1
1	1	0	0	0	0	0

Figure 7.3: Gates

7.2.2 Binary Numbers

Most people today use the Arabic Numeral System based on ten digits, which is a positional numbering system where the contribution of a digit to a number is determined by the digit and by its position in the number. So even though the '1' in '12' is a smaller digit than the 2, its contribution is larger because of its position. To be precise: there are 10 digits, so the largest single-digit number is 9. The next number can not be represented using a single digit, so two digits are used: the smallest non-zero digit followed by zero; 10. The 'weight' of the position of the 1 is equal to the number of digits (i.e. ten) so the total value is ten.

Using ten digits at hardware level is impractical, but using two digits does make sense: the digits 0 and 1. Binary also has a number 10, but the weight of the 1 equals now equals two (the number of digits).

Note the difference between a number and its representation(s):
This is a number of dots . . . .

- In decimal, the number of dots is written as: 12
- In binary it is written as: 1100
- In hexadecimal (which we will revisit) it's written as: c

7.2.3 Binary to Decimal Conversion

Conversion from binary to decimal is very simple.

- write down the weights of as many digits as you need
- for every digit 1, add the corresponding weight

For instance, to convert 1001 to decimal, write the weights: 8,4,2,1 (each position the weight is multiplied by two). There are ones at the first and last position so the value is 9.

Note that the weight of the n-th position from the right is 2^n, the n-th power of 2.

7.2.4 Decimal to Binary Conversion

To convert a decimal number to a binary number do the following:

1. if the number is even, write down a 0
2. if the number is odd, write down a 1 and subtract 1 from the number
3. divide the number by two
4. goto step 1 unless you have reached 0

The digits you have written down are the binary digits **from right to left**.

For instance: convert 91

`(91) 1 (45) 1 (22) 0 (11) 1 (5) 1 (2) 0 (1) 1 (0)`

Binary: `1011011` (check: 1+2+8+16+64=91)

Exercises:

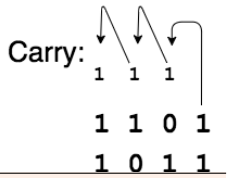
1. Convert the binary number 101 to decimal
2. Convert the decimal number 101 to binary

1. 1+4 = 5
2.(101) 1 (50) 0 (25) 1 (12) 0 (6) 0 (3)
1 (1) 1 (0) => 1100101

7.3 Binary Arithmetic

Exactly the same rules and methods exist for binary arithmetic as for decimal arithmetic: 1 + 1 can't be 2, so it is 10; that is 0 with a carry 1.

For instance, this is the addition of 1101 and 1011 (check: 13 + 11 = 24).



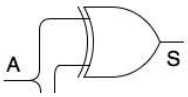
The tables of multiplication are trivial, so multiplication is a matter of rigorous discipline:

```
  110
  101
----- *
  110
11000
 ---- +
11110 (check => 2+4+8+16 = 30)
```

8. Components

8.1 Adders

A **half adder** is a component which adds the right-most bits of two numbers. It takes two inputs and produces two outputs: the sum and the carry. *Note how the component does the same as we do when we add two numbers!*



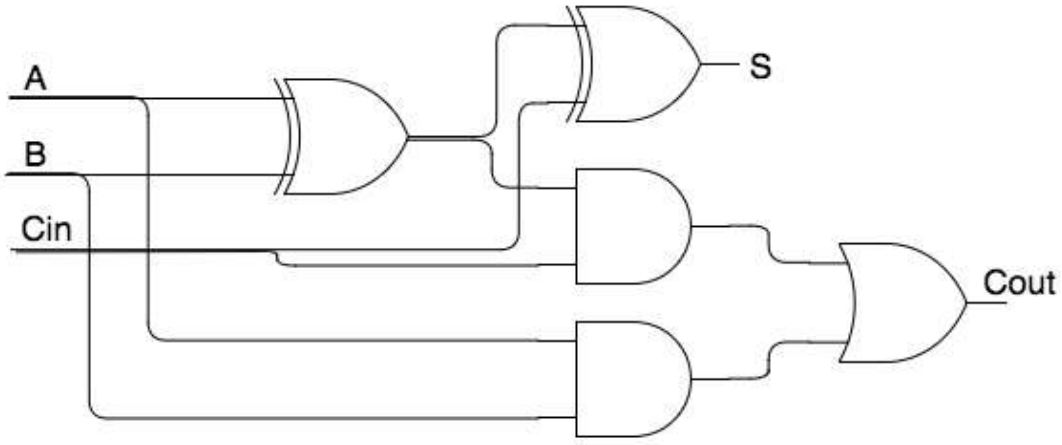


Figure 8.2: Full Adder

A ***full adder*** is a component which adds other bits of two numbers. It takes three inputs (two operands and the carry from additions to the left) and produces two outputs: the sum and the carry.

For obvious reasons trivial components such as one bit adders are often drawn as components (not showing the gates).

8.2 Larger components

From these parts larger components can be created. This is an N-bit adder, made from one half-adder and (N-1) full adders.

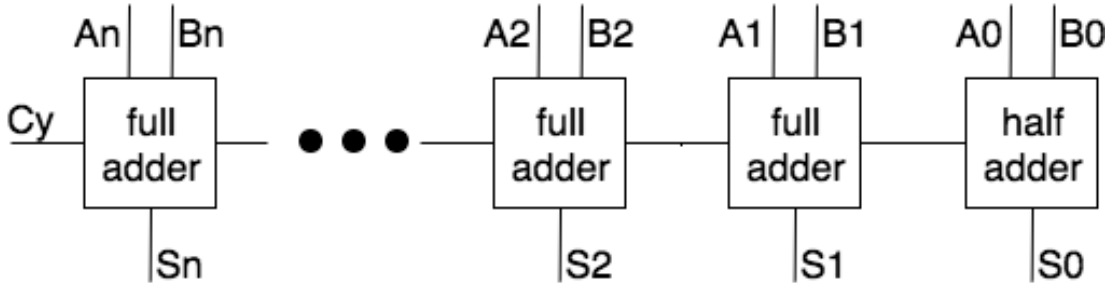


Figure 8.4: N-Bit Adder

In general, any component can be designed by breaking it down in Boolean functions, truth tables and gates.

8.2.1 Decoder

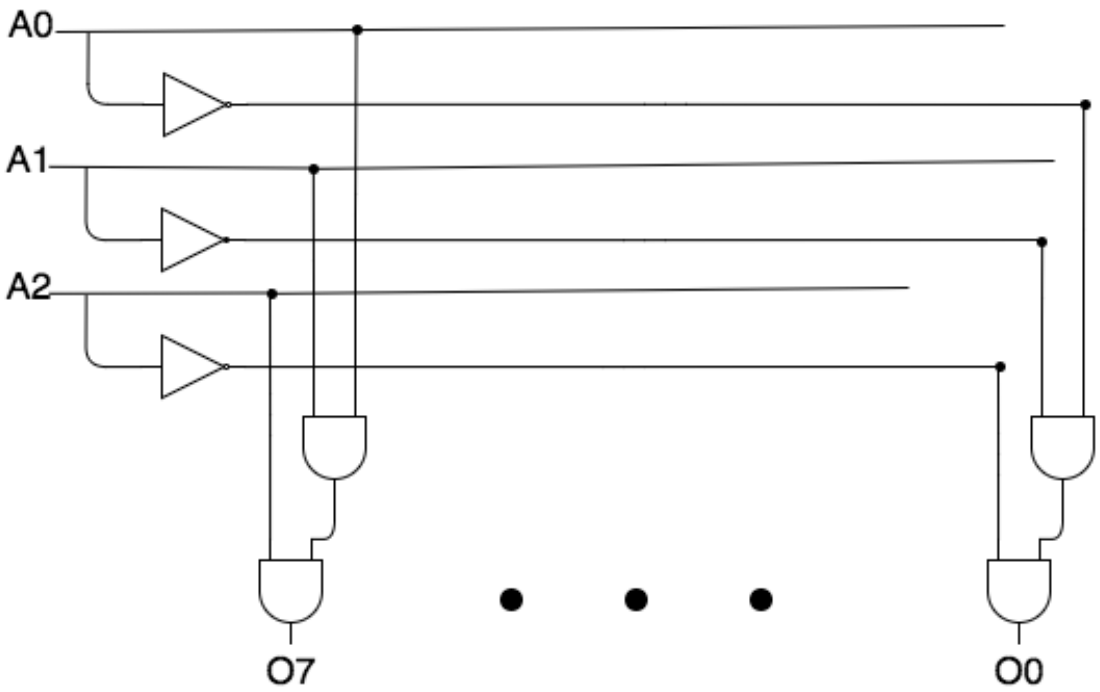


Figure 8.5: Decoder

A one-of-eight decoder uses three address bits to set one of eight bits on (and the rest off). For instance, `decode(1,0,0) => (0,0,0,1,0,0,0,0)`. Bits are numbered as usual from right to left starting with 0, so 100 (= 4 decimal) addresses the fifth right-most bit.

Each of the output bits turns on for one specific combination of input. For instance, `o(5) = i(1) AND NOT i(2) AND i(3)`. From this observation,

designing the circuit is straightforward.

- o(2), the high-order address bit, is only set if one of the high-order input bits is set, so:
  `o(2) = i(7) OR i(6) OR i(5) OR i(4)`
- o(0) is only set if one of the odd-numbered inputs is set:
  `o(0) = i(7) OR i(5) OR i(3) OR i(1)`
- And the last when odd pairs are set:
  `o(1) = i(7) OR i(6) OR i(3) OR i(2)`

The design using gates is now trivial.

An **encoder** does the reverse: given inputs one of which is 1 (and the others are 0) an encoder produces the address of the set bit.

- Design an 8 to 3 encoder

```
o = (a AND c) OR (b AND NOT c)
```

A multiplexer is an important component which uses one or more controls to copy one of two or more inputs to the output.

A 2-input multiplexer is implemented by the trivial equation for inputs a and b, control c and output o. The circuit follows trivially from the equation.

- Design a 2-input multiplexer.

This idea can be extended to

- more than 1 bit wide inputs
- more than two inputs (requiring more than one controls)
- 'output multiplexing'

*Note: a **demultiplexer** is a component which allows us to copy one input to two or more outputs, based upon controls.*

Note: a multiplexer can also be made using a decoder by AND-ing each decoder output with one input.

8.3 Flip-Flops, Latches

Not all components are Boolean functions. ***Flip-flops*** and ***Latches*** are components which maintain an internal state (0 or 1): they are one-bit memories. CPU registers are built from flip-flops and latches.

The simplest form is an SR latch, which has two inputs S and R and two outputs Q and <u>**Q**</u>. If S and R are 0, either Q or <u>**Q**</u> will be 1 (and the other 0) depending on the current state. When S is 1 (and R is 0) the state is ***set***. That is, Q will be set forthwith. When R is set (and S is 0) the state is reset (<u>**Q**</u>=1, Q=0). S and R should never be 1 at the same time because then the output will become unreliable.

Many other flip-flops and latches exist, such as a D-latch which has a control (C) and a data input (D). The internal state is set to D only when C is 1 and is unaltered otherwise.

8.4 Flip-flops & Latches => Registers

Flip-flops and latches (we'll discuss the difference later) can store a bit which can be read or changed using control lines.

Putting flip-flops or latches in a row of 8, 16, 32, 64 or more bits makes a logical component, which is called a **register** and which stores a value: an integer, a character, a float, an address.

9. Data

9.1 Positional Number Systems

As mentioned: numbers are an abstraction. One number can be **represented** in many ways, including the *positional number systems*.

Earlier we have seen that binary numbers are useful because electronics can directly represent them and perform calculations upon them.

For humans, large strings of ones and zeros are confusing at best. Who knows at a glance the significance of `11000000101010000000000011111111`?

For this reason humans use the decimal system with ten digits (originating from the fact that we have ten digits :-).

But still, long strings of digits aren't our strength: the number above is **3232235775** in decimal. Who knows its significance?

In this case the number consists of 32 bits, but by convention we are used to writing it as four decimal numbers separated by dots with each representing an eight-bit (one byte) section: `192.168.0.254`

For IPv4 this notation works, but it does have a disadvantage: each 8-bit section uses 1, 2 or 3 places in the decimal notation.

9.2 Hexadecimal

For this reason, **hexadecimal** is often used using the 16 digits **0-9** and **A-F**. Because 16 equals 2 ^ 2 ^ 2, a 4-bit number (which is called a **nibble**) can be written using one hexadecimal digit, and a byte as two digits.

Now, long numbers can be written in a way which humans can copy at a glance. For instance, the IP (v6) number of Google's primary DNS server is: 2001:4860:4860::8888 (note: :: means all zeroes). Try it out in your browser: `https://[2001:4860:4860::8888]`.

Other places where hexadecimal is used extensively include **memory dumps**. Memory is usually divided in bytes (8 bits) and then records (disk) or pages (memory) of 512, 1024 or 4096 bytes each (2^9, 2^10, 2^12). Memory addresses on a modern computer are typically 64 bits.

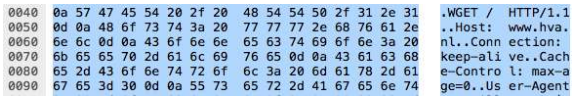


Figure 9.1: Wireshark

This screenshot is part of a Wireshark dump of an HTTP GET message for hva.nl. Most memory dumps look similar.

The left column consists of the offset within the package, the middle part shows the content in hexadecimal, and the right column the character interpretation (because humans read this more easily when it represents text).

Because memory dumps are hexadecimal, specialists using them must be able to convert hexadecimal to decimal to ascii almost on sight.

9.2.1 Hexadecimal to Decimal

One nibble: digits 0-9 are themselves; digits A-F or a-f => 10-15

One byte: 16 times the first nibble plus the second. E.g. f9 => 15*16+9=249

Larger numbers: per byte or nibble, and multiply each by a power of 16 (nibble) or 256 (byte). E.g. A4c9 = (10 * 16 + 4) * 256 + (12 * 16 + 9) = 42185

Such a calculation can be made using pen and paper.

9.2.2 Hexadecimal to Binary and Vice-Versa

One nibble: learn the table by heart.

```
0: 0000    4: 0100    8: 1000   12 C: 1100
1: 0001    5: 0101    9: 1001   13 D: 1101
2: 0010    6: 0110   10 A: 1010   14 E: 1110
3: 0011    7: 0111   11 B: 1011   15 F: 1111
```

Larger number: just string the digits together. A4c9 => 1010 0100 1100 1001

Similarly, the reverse is easy:
101 1100 1001 0100 1010: 5C94A

9.2.3 Decimal to Hexadecimal

If you are comfortable doing long division: repeated division by 16 noting down the remainders does the trick.

```
50109 / 16 = 3131 rest 13 (D)
3131 / 16 = 195 rest 11 (B)
195 / 16 = 12 rest 3
12 / 16 = 0 rest 12 (C)

50109 = 0xC3BD
```

If not, conversion to binary is easiest, noting down the nibble values using the table.

9.3 Two's complement

One of the most effective System Engineering feats is **two's complement**: a representation of negative integers which allows the same digital electronic circuit to perform addition of negative **and** non-negative integers as well as subtraction of those numbers!

Naively, negative integers might be represented by using one specific bit to represent the sign. For example, 00000101 might represent 5 (in one byte), and 10000101 might represent -5. There are two problems:

- Now there are two zeros: 00000000 and 10000000. Since zero is probably the most-used number, having to check *which* zero we're looking at introduces significant overhead.
- addition and subtraction don't work immediately 00000101+10000101=10001010 (-10) but should be 0. This means this representation requires different circuitry for addition and subtraction

In two's complement a negative value is represented as the complement when subtracting it from the next larger power of two. So: for 8 bits the complement of a number n is 100000000-n (8 zero's, nine bits). E.g. -5 is 100000000-101 is 11111011 is 0xFB.

In this representation the leftmost bit does indeed represent the sign 1 => negative, 0 => zero or positive. Also, there is only one zero: 00000000.

But most importantly, the rules for addition work whether a number is negative or positive:

```
11111011
00000101
--------8
00000000 (= zero in 8 bits, as it should be)
```

9.3.1 Conversion

Consider this 4-bit addition (we'll use 4-bit numbers to keep the examples small):

```
1011 + 0101 = 0000
```

Is the first number positive (11) or is it negative (-5)?

You can't tell!. The rules for addition are identical for positive and negative numbers, so the context decides how this should be interpreted.

In general data are just strings of bits; interpreting them without context is impossible.

We've explained how the two's complement can be computed by the subtraction from the next power of two, but there is an easier way to do it:

1. flip the bits (1 <=> 0)
2. add one

So: 00000101 => 11111010 => 11111011

Interestingly, the other way round works exactly the same!

9.4 Other Data

What does the number `10111010101011011111000000001101` signify?

9.4.1 Information $\neq$ Data

It's a trick question:, you can't know. It's just data, but without context it's not information.

- It might be an IPv4 address: 186.173.240.13
- It might be a very large integer: 3131961357
- Or a negative integer: -1163005939
- It might be a very short text: `(*$\degree-\eth$*)` (degree symbol, minus sign, greek eth if it is marked up wrong) followed by a newline
- Or it might be a memory pointer. Who knows what's there.
- Further down we will see that binary numbers can also represent floating point (approximate real) numbers. This number is in fact -0.0013270393.

Would you believe this number (-0.0013270393) has several hits on Google?

It is a Microsoft error message indicating a corrupt heap; convert to hex to get the joke!

9.5 Floating Point Numbers

IEEE standard 754 is a standard for the representation of approximate real numbers:

- the largest numbers used in sciences (say the number of atoms in the universe)
- the smallest positive numbers used in sciences (say the weight of an electron)
- integers precisely (not approximately)
- common operators (addition, comparison) can be easily done in hardware

And we use only 32 bits for a number.

9.5.1 Scientific Notation

Humans are bad at long strings of numbers. What would you rather get: €1341265142643 or €924395963572?

Scientists have long known this and have come up with a notation which improves upon this 1341265142643 = 1.34 x 1000,000,000,000 = 1.34× 10^12 = 1.34e12

This 'scientific' notation has two advantages

- Looking at the power of ten offers an immediate impression of the scale of the number (in the example above, 1.3e12 vs 9.2e11; the first number is bigger).
- As many digits can be shown as are useful given the circumstances.

In the notation 1.34e12 the number 1.34 is called the **mantissa**, and 12 the **exponent**. In calculators the number e is often used: 1.34e12.

Note that the exponent can be either positive or negative. 10^-3 = 1/(10^3) = 0.001. So 1.34e-2 = 0.0134

9.5.2 Calculation

Calculation is straightforward.

- In *multiplication*, the mantissas are multiplied while the exponents are added. 2e2 x 3e-3 = 6e-1
- In *addition*, the exponent should first be made the same and then the mantissas can be added: 1.2e4 + 2.2e2 =
  - 120e2 + 2.2e2 = 122.2e2 or
  - 1.2e4 + 0.022e4 = 1.222e4 or indeed
  - 12e3 + .22e3 = 12.22e3

9.5.3 IEEE 754

The scientific notation is used as a basis in IEEE 754 with an obvious adaptation: since we are using binary numbers, the base for the exponent is 2.

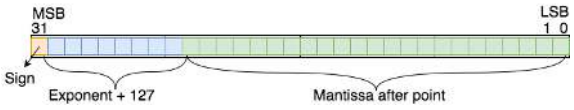


Figure 9.2: Floating Point Numbers

Note that every binary number except zero contains a one (either before or after the decimal point). This implies that every number (except 0) can be **normalized** by shifting it left or right until it is written as 1.×. For example, the number 0.0000101, which is 5/128 can be written as 1.01e-5

Since the first digit of the normalized mantissa is always 1, it isn't stored.

The exponent can be positive or negative. We could store it as a 2's complement number, but in fact we simply offset it by 127. A typical engineering solution: this way sorting (the > operator) is the same for ints as it is for floats.

For instance the representation of 0.375 is
- it's positive, so first bit 0
- 0.375 = 0.25 + 0.125 = 1/4 + 1/8 = 0.011 (bin) = 1.1e-2
- exponent bits 125 binary: 01111101
- mantissa bits: 100×0 (the first 0ne is dropped)
- 001111101100000×0

9.5.4 Conversion to Decimal

What floating point number does 0x40490fdb represent:
- binary: 01000000010010010000111111011011
- sign: positive
- exponent: 10000000 (128) - 127 => 1
- mantissa: 1.10010010000111111011011
  - Note: working with fractions of 2 soon becomes error prone. An alternative is to view this as a large integer (convert to hex) and divide by 2^23. So: 1100 1001 0000 1111 1101 1011 = 0xc90fdb = 13176795.

9.6 ASCII

At their heart, computers process zeros and ones. But using positional numbers we can process all non-negative integers; using two's complement we can process negative integers; and using the floating point representation we can process (near) real numbers. In short, we can process most 'kinds' of numbers.

But, for the moment skipping video and audio, there is another very important type of data we need to process: text.

The American Standard Code for Information Interchange (ASCII) is an ancient standard which plays an important role to this day. ASCII represents many common characters in numbers.

ASCII was created around 1960, in a time when the human-computer interface consisted of a teletype (which is a combined keyboard and printer). At the time, a 7 bit code was sufficient to represent all common letters (upper and lower-case), numbers, punctuation and necessary control codes to manage the teletype. For instance, ASCII still has a code to ring a bell in the teletype to get an operator's attention when a message ends. Also, ASCII has a code for carriage return (the carriage is the small train that carries the printer head which hammers letters through an inked cloth ribbon) and for line-feed (which advances the scroll of paper to the next line). Whenever we embed a **\n** in a text to get a newline we are inserting a line-feed symbol.

Today we use Unicode to encode just about all known symbols, and we use encodings such as UTF-8 to pack unicode characters in single or double bytes. However, the ISO 646 subset of UTF-8 still coincides with ASCII and ASCII is still the preferred representation when we happen to be limited to plain text in the English alphabet.

The well prepared programmer can somewhat read (hexa)decimal data which represents text. Look up an ASCII table and you will see that there is logic to it. In hex:
- 00 and onwards are the control characters including NUL, BS, TAB, LF, CR and ESC
- 20 and onwards are the space and punctuation
- 30 and onwards are decimal digits
- 40 and onwards are @ and capital letters
- 60 and onwards are backquote and lower-case letters

Using this information you can decipher `49 20 41 4d 20 4E 4F 31`

10. Mechanisms

10.1 Memory

In principle, memory is capable of storing a large number of bytes. Every byte stored has a unique location called an **address**. If a laptop has 16GB of memory, the memory can hold 16*1024*1024 bytes. To address that many bytes there must be that many distinct addresses, so every address is (at least) a 24-bit number.

A register is made from flip flops or latches, each of which consists of at least four transistors. Dynamic Ram is an improvement which requires only 1 transistor per bit. The downside is that every bit must be read and written frequently, or it would degrade. DRAM chips have a circuit that refreshes all bits as long as they are powered. That is why normal RAM in your computer is emptied when you turn off the power.

Memory has three inputs and one output.

- One input is the **control**. It determines if we want to store data or fetch stored data
- The second input is an **address** where to fetch data from or where to store data (depending on the control)
- If the control says 'store', the third input is the byte to be stored. Otherwise this input isn't used.
- If the control says 'fetch', the output will be the byte that was last stored at the given address. Otherwise the output isn't used.

(*In practice, bytes aren't fetched or stored one at a time, but in larger chunks (2, 4, 8 or much more*).

Loosely, fetch means the following:

- the CPU computes an (integer) address
- puts the address on the **address bus**
- sets a **control line** to indicate if it wants to **fetch** the data or **store** it
  - store: the CPU puts the appropriate value on the **data bus**
  - fetch: the CPU must *wait awhile* before it can read the value from the data bus

A lot of engineering concerns optimizing this naive approach. Nobody likes CPU's to just wait.

10.2 Address ≠ Data

It is important to understand that an address is 'just data' until it is put on an address bus. Computing an address is just an integer computation.

For instance, to execute `A[5]=1024` if `A` is an array in which each cell contains a 64-bit integer, then the address of cell `5` must be computed as follows

- get the current location of the first cell *&A*
- add to that the size of cells times the number of cells to skip
- so the location of field *A[5]* is address `&A + 4 \* 8`

10.3 Registers

Although memory is fast, it is still too slow compared to the ALU and core operations. To achieve the best speed, **registers** are used. *From a programmer's perspective, memory resembles an array and registers resemble local variables.*

There are two categories of registers: **special purpose** and **general purpose registers**. General purpose registers are used to compute and hold various results relevant to a program; special purpose registers hold specific values relevant for the CPU. For instance, the flag-register holds all status flags resulting from computations in the ALU (including Negative, Zero, Carry).

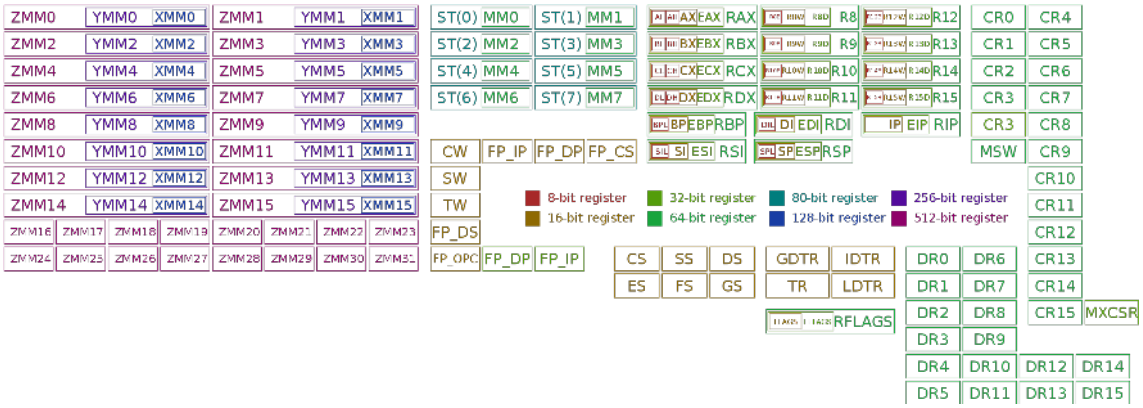


Figure 10.1: x86-Registers

Source: 'Immage - Own work, CC BY-SA 3.0, https://commons.wikimedia.org/w/index.php?curid=32745525'

10.4 Fetch-Decode-Execute

A low-level program is stored in memory as a sequence of instructions. An processor performs a fetch-decode-execute cycle:

- fetch instruction from memory
- decode to see which operation should be performed on which operands
- execute

A 6 minute overview can be found here)

10.5 Program Counter + Instruction Register

When we explain programs, we point at the code and say 'here it does this, and here it does that'. An executing program has what is called a **locus of execution**: a point in the program which describes what should happen at some point in time.

A processor keeps the address of the next instruction (its 'locus of execution') in a register traditionally called the **program counter** (PC – note that it doesn't count anything).

Instruction fetch means:

- get the byte or bytes (many instructions require multiple bytes) pointed to by PC
- put the instruction is a special **Instruction Register** or Opcode register (IR, OPC) ready for decoding
- advance PC (ready for the next cycle)

General purpose registers allow for many different kinds of computations. For instance, the ARM instruction `SUBS R8, R8, #240` takes the contents of register R8 and subtracts the value 240 from that, storing the result in register R8.

Treating the PC as a general purpose register makes sense because, **many 'general purpose' operations are meaningful in this context**.

For instance, what is the effect of the operation above if `R8` is in fact the program counter? If the instruction is fetched from location *here*, then the next instruction will be fetched from location *here-236* (next instruction after *here* minus 240).

In ARM this is the **Branch** instruction, in this case with a negative offset. *The result is a loop.*

A listing of x86 instructions can be found here and an in depth description of real ISA instructions can be found in Intel's programmer's manual (Intel® 64 and IA-32 Architectures Software Developer's Manual) which lists all instructions including data transfer, arithmetic, logic, control transfer (affecting the program counter) and much much more.

10.6 Autoincrement

When an instruction is fetched, the PC is immediately incremented. This doesn't take additional time; it is done by hardware in the same machine cycle.

This is an example of auto increment, which is used frequently, for example when using **stacks**.

10.7 Stacks

Functions are an important concept in programming. When we call a function, the current locus of execution is suspended, and we continue execution (point the locus) in the function body. When it is done, the previous locus is restored, and execution continues.

The same concept is used in a CPU: a **call** instruction stores the current PC and then loads a new value into it (the location of the function body).

The PC is stored in a chunk of memory called the **stack** which is pointed to by a register (sometimes called SP). Storing the PC uses autoincrement (or -decrement) on SP, and the **return** instruction does the reverse.

Functions have parameters and local variables. Where are they stored? Maybe in registers, but what if I call another function? Then my local variables must be stored in memory so that they can be retrieved when the second function returns.

A stack is also used for this. All parameters and local variables are stored in what is called a **stack frame**, and the stack pointer is advanced to empty space beyond the stack frame.

A special register (frame pointer) and pointer trickery is used so that the CPU (which doesn't really know functions) can find the beginning of each stack frame.

10.8 Interrupt

When you move the mouse on your computer, that movement must be dealt with immediately. An interrupt stops a CPU core and briefly executes code to update the mouse pointer. Then the core is allowed to continue what it was doing before. A special function is activated to handle this mouse

event. When that is done (perhaps a micro-second later), the core must resume what it was doing before, without any of its registers having been altered.

But what was it doing? The context of what it was doing was stored in memory, on a **stack** using the same mechanism as described earlier for function calls: registers are stored at the location pointed to by an SP in a frame.

10.9 Tri-State

What happens when two outputs are connected to the input of a third component. If they are both 1, do they add up and become a 2? If they conflict, does the CPU short circuit and explode?

In some technologies, **tri-state** outputs exist, which are 0, 1, or a third, floating state. That state can be connected to a 1 or 0 state and won't influence it.

In other technology, a 0 isn't grounded so connecting it to a 1 just results in a 1 output. They are OR'd together.

In modern CPU's tri-state is only needed on external ports so that your CPU doesn't fry when you insert a USB stick the wrong way.

10.10 Buses

Connecting the output of one component with the input of another is simple: just put a wire or metalized path in between. Connecting multiple inputs to one output is also straightforward, using a rake-shaped path. But connecting multiple outputs to one or more inputs poses a problem. What if the outputs disagree? If different values feed into one input, essentially arbitrary values are read.

A **bus** is a 'device' which solves this. In addition to the rake-shaped path it has a gate and a control line to every output. If its control is 0, an output doesn't contribute; only the gate for which the control is 1 can output a value, which is then passed on the entire rake.

The control circuitry ensures that at most one output is passed on to the bus at all times.

10.11 Delays

How fast can a processor go? Transistors switch incredibly fast, but still require more than zero time. What limits the overall speed?

- Physical speed. Today, a CPU clock speed is about 3GHz. Time between two ticks is about a third of a nano second. In that time, any signal (limited by the speed of light) can travel about 10 cm. Since processors are tiny compared to the distance light travels even in these small time slots, signal traveling time isn't likely to be the limiting factor (for now).
- Transistor switching speed. A signal is a current which must load a capacitor (such as the gate in a transistor) and overcome resistance in order to switch. But the switching time of modern transistors is on the order of picoseconds. Single transistor switching time isn't the limiting factor.
- Sequencing. Consider the N-bit adder consisting of a half-adder for bit 0, and N-1 full adders for the remaining bits. Each full adder takes the carry of the previous addition and uses it in its own calculation. In a 32-bit ripple-carry adder the tiny delays before each carry is valid adds up. (Note that carry-lookahead is a typical system-engineering approach to reduce this delay)

The tiny delays become a problem when considering stateful components such as latches. When a latch is set its input should be valid, but how is the latch to know?

10.12 Clocks

A clock is a crystal-based component that gives a pulse every so many nanoseconds Components use the clock to know when they can use the output of other components.

Circuitry is used off of a clock to create other clocks:

- A delay creates a clock with the same frequency but different phase
- A divider creates a clock with lower frequency
- A multiplier creates a clock with a higher frequency. Most processors use a clock of a few hundred MHz with a multiplier to reach their desired running speed

A clock grid transports the clock pulse across the entire CPU so that all components can use it with the least possible delay.

10.13 Flip-flops & Latches Revisited

A ***flip-flop*** is changed when the control line is high. A ***latch*** is changed only when the control changes, from low to high. The purpose is the same (storing a bit) and the context determines what type flip-flop or latch is appropriate.

The ability to choose allows us, for instance, to perform a calculation when the clock is high, and to capture the result when the clock changes to low.

Clocks and precise timing are outside the scope of this writeup.

Part Three

11. Bigger Things

So far we've looked at the lowest level of Bits & Gates, but now we're off to bigger things.

Designing a CPU in terms of billions of transistors or gates as logical units would be like designing a city in terms of sand, wood and glue: it doesn't work that way.

We use transistors and gates to design bigger and bigger functional components, which, together, form the functional parts of a CPU.

11.1 Data Path

A ***data path*** is the part of the μarchitecture concerned with data and computation. It consists of registers, buses and the ALU. During every cycle, controls determine which registers output data on buses A and B, which function the ALU should perform, and to which registers the result will be stored through bus C.

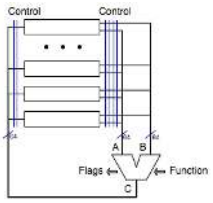


Figure 11.1: Data Path

11.2 ALU

Probably the most complex component part of a CPU is the ALU: the ***arithmetic and logic unit***, i.e. the unit that does all the computations.

We have already seen many parts:

- Adders for integer addition (and as we've discussed two's-complement it is clear this is also subtraction)
- Other gates for logic functions
- Decoders for register selection
- Multiplexers for input selection
- Multiplexers for function selection (see image)

An important system engineering method: a control bit alters the function of the circuit. This is the smallest possible interpreter! We will look at the ALU later on.

According to the *von Neumann model* a CPU consists of two aspects: control and ALU. An ALU computes: integers, other numbers, logic: AND, OR, etcetera. In principle, an ALU has three (multi-bit) inputs and two outputs:

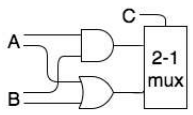


Figure 11.2: Multiplexer

- Input: The desired operation. ALU's can do many, so control lines make the ALU work this or that way.
- Inputs: The operands. Often two (+, *,&, ...) but sometimes one (!, -)
- Output: the result of the operation.
- Output: Flags. In addition to the result, the ALU will offer more information about the operation. This information can subsequently be used (by the control unit) to perform additional action. These include =>
  - ***Zero***. Whether or not the result of a computation is 0 is important to decide subsequent action. For instance, we loop through an array from end to start, which is index zero.
  - ***Negative***. Likewise, whether the result of an addition or multiplication is positive or negative is an important decider.
  - ***Overflow***. When two positive 32-bit integers are multiplied the result must be positive. If it comes out negative (i.e. the left-most bit is 1) the product is clearly too large to represent in 32 bits two's complement. This is called *overflow*, and an error handler must be activated.
  - ***Carry***. The carry is relevant output, for instance when considering what is called 'high precision arithmetic'. If 32 or 64-bit operations aren't sufficient, higher precision addition can be achieved using multiple additions in series, making sure that the carry of the lower order is passed on to the higher order.

In the next few sections we will create a simplistic ALU using components we've seen so far.

11.2.1 Boolean Logic

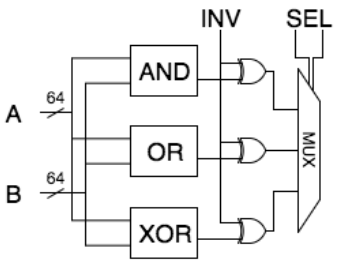


Figure 11.3: Boolean Logic (1st try)

- We'll start with Boolean logic, and let's say we need at least the common operators: AND, OR, XOR. We've already seen these.
- Adding components for NAND etcetera is extravagant but actually we only need a NOT behind the AND which we can turn on or off.
- After that we select the result we need (MUX).

Oh wait, we forgot the operator NOT. We can NOT the result of the binary operators, but we can't currently compute NOT A

Since we already have the inverter pattern and since we're not using one of the four selection inputs, the solution is straightforward. Add an 'operator' which just copies A and ignores B:

That's our logic unit. With three control bits we compute the relevant functions AND, OR, XOR and NOT and throw in NAND, NOR, NXOR almost for free (and "Copy A" (A,B) => A, which isn't particularly useful).

11.2.2 Arithmetic

We've already seen a binary adder (consisting of a half adder and a row of full adders). As mentioned, two's complement allows us to add unsigned/positive and negative numbers using the same circuit.

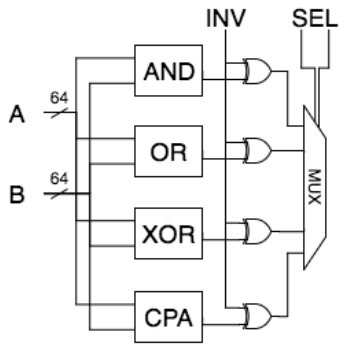


Figure 11.4: Boolean Logic

Creating a separate circuit to perform subtraction is in fact not necessary: *subtraction is the same as addition of the negative of that value: a - b = a + (-b).*

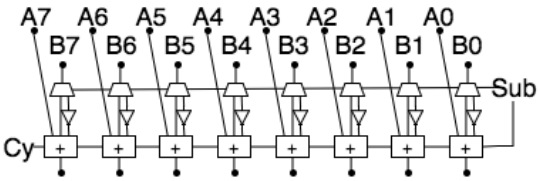


Figure 11.5: Add/Subtract Integers

Computing the negative of a number is easy: flip the bits and add one. Adding one may seem to require a separate cycle, but ***it is in fact very much like an additional carry***. By changing the half-adder to a full adder, we can add one or not depending on the operation. Our arithmetic unit (shown only for 8 bits) now looks like this.

11.2.3 Shifters

Shift is an important operation on binary values. Use cases include processing each bit in a value, for instance for transmission (Networking Layer 0), or very fast multiply/divide by powers of two. Shift left = times 2. Also, many cryptography algorithms use shifts. Common operations are:

- LSR: Logical shift right, copying the rightmost bit into the carry (to be tested and used).

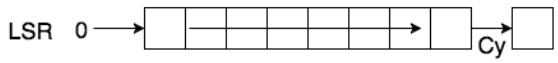


Figure 11.6: Logical Shift Right

- LSL: Logical shift left (also ASL). This is times 2 for unsigned numbers.

Figure 11.7: Logical Shift Left

- Arithmetic shift right keeping the sign-bit in place so that divide by 2 is valid (rounds down towards negative infinity).

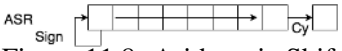


Figure 11.8: Arithmetic Shift Right

- Rotate left.

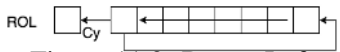


Figure 11.9: Rotate Left

- Rotate right through carry.

11.2.4 Barrel Shifter

Shifting is a very much used operation, for instance in the context of address calculations. Usually we need to shift more than once, and sequential shifts (i.e. in a loop) take many cycles.

A ***barrel shifter*** is a component that can shift an arbitrary number of times in a single cycle. The diagram shows an 8-bit (logarithmic) barrel shifter. It uses three-bit input B to determine how many positions we must shift (000-111, zero to seven in an 8-bit shifter).

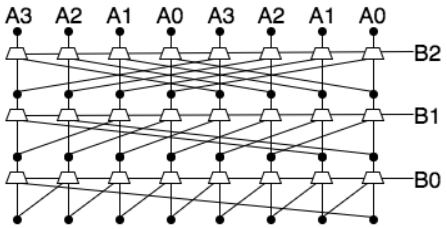


Figure 11.11: Barrel Shifter

Note how the MSB of B (B2) controls the top row of demuxers, connecting the output either vertically down, or 4 places to the left. And note that when rotating, to the left of A7 is A0, and so on.

The lesser significant bit B1 controls the second row, where each bit is transported either down or two places to the left. And finally the LSB B0 controls the last one or zero shift.

For instance, to shift A five positions, B2 and B0 are set, so first the bits are shifted four to the left and then they are shifted once more, each time rotating them from the left (MSB) to the right (LSB) as necessary.

We've now seen all major components in an ALU and we've seen the mechanisms to join them into one large component (of less than one millionth of a millimeter squared).

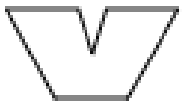
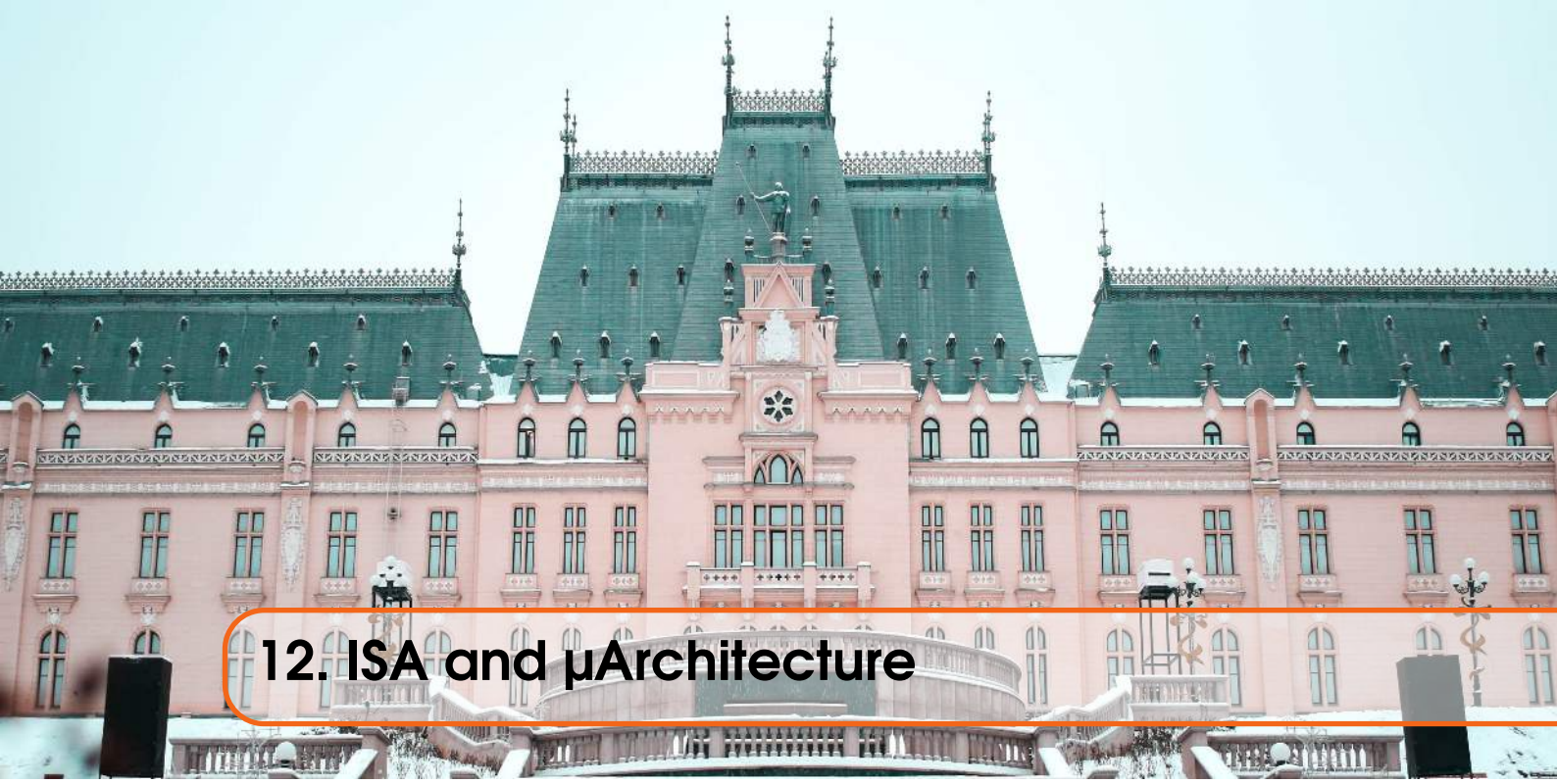


Figure 11.12: ALU Symbol

The components we've seen consist of

- A Logic unit offering NOT, AND, OR, XOR, NAND, NOR, NXOR
- An arithmetic unit offering ADD and SUB
- We've shown a barrel shifter offering ROL. A barrel shifter which performs ROL, LSL, ROR, LSR and ASR is an extension of what we have already seen.

A 'real' ALU does much more, including multiplication and division. See the Intel® 64 and IA-32 Architectures Software Developer's Manual to be astounded.



12. ISA and µArchitecture

Around 1950, as the demand for more powerful, faster processors was growing, one engineer, faced with the increasingly complex and error-prone task of designing new processors, had a great thought: create a simple, more reliable 'CPU' which simulates more complex processors.

Chip manufacturers need to innovate already complex CPUs while computer designers, operating system vendors and compiler developers continue to use and expand upon existing software for the 'older' CPUs.

- An Instruction Set Architecture is an abstract model of a CPU. Software can be written for the abstract model, and any actual CPU that implements (i.e. simulates) the model will be able to execute the code.
- A microarchitecture is the most complex digital logic component. Its purpose is to simulate the ISA. You may know some µArchitecture by name: Intel: Haswell, Skylake, Golden Cove, Snapdragon: Kryo, Nvidia: Denver.

In order to understand the microarchitecture, it is necessary to understand its context: the ISA it must emulate.

An ISA is a model of a 'virtualized' CPU, realized by a microarchitecture. The programmer's model offers the following aspects:

- The CPU has registers, some of which have a special purpose (such as the PC, IR and SP) and some of which are general purpose.
- a program consists of a list of instructions pointed to by the PC; every instruction consists of one or more bytes
- The CPU executes a fetch-decode-execute cycle in which the instruction pointed to by PC is fetched, it is decoded and operands are fetched
- Then it is executed, for instance by the ALU, and possible results are stored

Perhaps the two best known ISA models are **x86** and **ARM**.

- **x86** is a family of models dating back to 1978, when it was taken from the then advanced microprocessor 8086. Since then, the model has been extended, but always remained backward compatible to this day. All attempts to bring out a CPU which isn't backward-compatible with x86 have been rejected by the market.

- **ARM** dates back to the 1980's to the British Acorn computer and was based on the 6502 processor. ARM has been used by Apple and many other mobile vendors and is still used because of its low energy requirements.

About twenty years later when computers started to be used more and more in businesses and education, system engineers were again hard pressed to improve the designs of their CPU's and other components to create more and more powerful computers. Not only were CPUs required to be faster and bigger (i.e., more on-chip memory), but the advances in programming language design also required new and complex capabilities to be added.

Designing functionality in chips is intricate and time-consuming so mechanisms were sought to make this faster and more manageable. The limits of the possible were found while trying to create hardware that executed instructions implementing complex programming patterns such as array access.

Two different philosophies emerged:

- Today we call computers built until 1970 ***CISC***: Complex Instruction Set Computer. The statement *a[i++]=o.b+2* might be compiled to a single instruction on a CISC processor. However, that instruction took many clock cycles to execute.
- On the other hand, a ***RISC*** (Reduced Instruction Set) computer offers simple instructions, but requires multiple instructions to implement such a complex statement. Because instructions and addressing modes are simpler, the chip design is simpler.

By itself, RISC isn't faster than CISC (just as many clock cycles are needed), but making instructions transparent and modular did allow for faster development and more optimizations around the ALU.

Today all CPUs are RISC by nature but the instructions that are supported are as complex as the CISC instructions of the 70's. Many simple instructions are executed directly by the (hardware) µArchitecture, but more complex instructions can be simulated.

Today, we use a language such as Verilog or VHDL to describe the circuits, and then use computers to create chips from that description.

12.1 Micro Architecture

A Microarchitecture (µArchitecture) is a hardware processor, the purpose of which is to simulate an ISA. Key processor generations are named after the µArchitecture that drives them. The ISA may contain complex instructions which are interpreted by the µArchitecture (in this sense, ISA is CISC). Chip manufacturers can innovate CPU's by creating new µArchitectures, as long as the µArchitecture implements the standard ISA. Note however, that the ISA itself can also be upgraded (usually backwards compatible, as mentioned). Today's x86-family extends over many generations, for instance growing from 8-bit registers to 64-bit registers

The µarchitecture is the most complex component that can still be understood in terms of underlying hardware (transistors).

The main purpose of the µarchitecture is to implement the ISA. That is, the ISA may offer functionality which is simulated by the μarchitecture either by direct implementation in μarchitecture instructions, or by simulation of one ISA instruction by multiple μarchitecture instructions.

The fact that the µarchitecture is driven by software is occasionally used to alter the ISA interpreter, for instance when there are bugs in the hardware, or to prevent vulnerabilities such as Specter.

12.1.1 Mic-1

In Andrew Tananbaum's wonderful book 'Structured Computer Organization' a microarchitecture called Mic-1 is presented, which can be easily understood with all aspects we have seen so far. It is

designed for educational purposes, but allows us to zoom in on many relevant aspects.

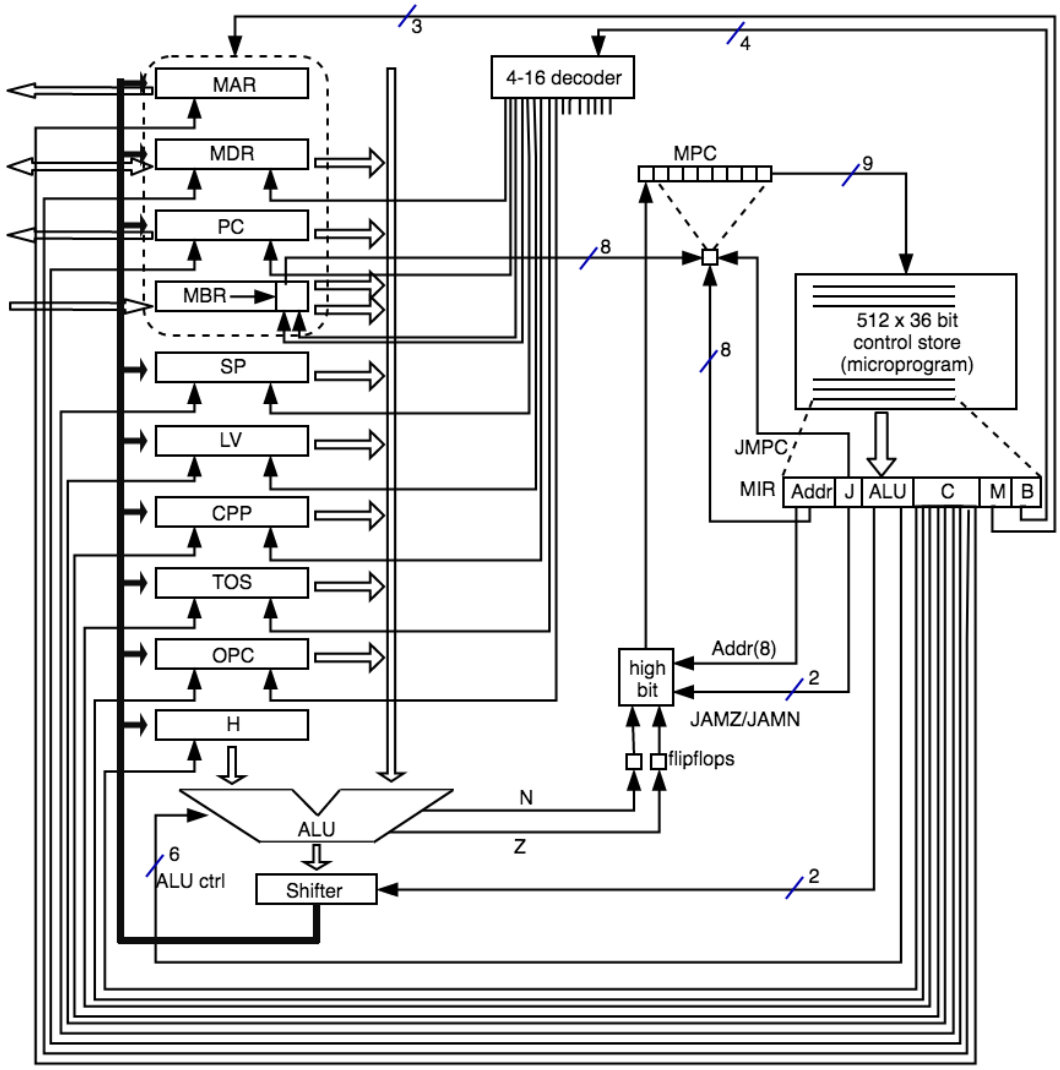


Figure 12.1: x86 Registers

Briefly:

- Mic-1 has ten 32-bit registers.
  - PC is the ISA program counter
  - MAR, MDR and MBR are used to communicate with memory (see below)
  - SP and LV are the stack pointer and pointer local variables (stack frame)
  - CPP contains a pointer in memory to all constants in a program
  - TOS is the top-of-stack, kept in a register to gain speed
  - OPC is the ISA instruction register
  - H contains intermediate results
- Program Counter (MPC) points into memory. Every cycle 36 bits are copied to MIR (fetch) to drive the µArch (decode)
- Different fields in MIR drive different parts of the µarch
- Two registers are loaded into the ALU. To keep Mic-1 simple, one is always H (half-result).

For instance, 5*6+7 would be computed first by loading 5 in H, then multiplying H with 6 (from memory), writing the result 30 in H, and then adding 7 to H.

- So precisely one register must be loaded through the B bus into the ALU every cycle. Therefore the B field has an index which is decoded to one of 9 registers.
- The result of a computation can be loaded in more than one register, so the C field isn't decoded, but controls 9 registers independently to accept the value from the C-bus
- The ALU accepts 6 control lines and 2 lines to an (independent) shifter. 6 bits allows for 64 operations in total, which include and, or, xor, not, +, -, *, /, increment, decrement, etcetera
- The shifter allows an additional shift within the same cycle, which allows (among other things) to cycle conveniently through all bits of a value
- The N and Z outputs reflect if the result of a numeric computation is negative or zero, respectively. These results are stored in flip-flops (memory) to be used in the next instruction. In this way conditional jumps (if, for, while) simply use this status
- Unlike ISA instructions, every µinstruction contains the address of the next instruction and JAMZ and JAMN bits to determine if the N and Z flags will be used (i.e., a conditional jump). If so, the high bit of the next address is (re)set. This means both the true-part and the false-part are reached without additional jump!

Mic-1 communicates with memory in two ways: 8-bit data or 32-bit words. The first is used to fetch ISA instructions: the ISA PC is used as address to fetch one byte into the low part of MBR. While that is fetched, the PC is incremented in one cycle. MAR is a word address, meaning that four bytes of ISA data are fetched or stored at once to/from MDR. Two control lines signal memory read/write through MAR/MDR; one control line signals memory read through PC/MBR

12.2 Conclusion

Starting from the humble beginnings of the transistor we have now been able to sketch the working of a µArchitecture, and through that hinted at the workings of modern processors.

Several aspects haven't yet been looked at. We will discuss

- Language Processing
  We have used the word '**interpretation**' loosely, but we will briefly go into language processing in general;
- Parallelism
  In order to achieve ever greater performance, CPU's continue to be engineered. The most important manner to do so is to device ways in which more computation can occur at the same time, by parallelization;
- Virtual Memory
  Regarding memory as an array of bytes is an oversimplification;
- Multi-tasking
  Finally, how can processors execute thousands of programs seeming at the same time, and how do they make use of the huge storage at their disposal

13. Data Types

A modern ISA understands several data types (implied by an instruction):

- Bits
- Bytes
- Ints (32 or 64 bits, whichever fits in a computation register)
- Short and long ints (16, 32, 64, 128 bits) not normally part of an ALU computation but supported in various instructions
- Pointers (i.e. addresses)
- Instructions
- Others: floats, strings, very long ints (currently 512 bits), vectors, ...

13.1 Word

A ***word*** is a number of bytes that fits in a compute-register and on an ALU bus, which is often also the width of the data bus. Bytes and words can be fetched and stored in one instruction. Often other sizes can also be fetched and stored in a single instruction.

The order of bytes in a word, and therefore in memory, may differ between systems, though for obvious reasons they are the same in one system. Named after Gulliver's travels they are called Little Endian (least significant byte has lowest address) and Big Endian.

CS experts must be aware of this possible difference to interpret memory dumps.

13.2 Addressing Modes

The CPU is connected to memory with an ***address bus***, which is usually 16, 24 or 32 bits wide. If the bus is narrow, few addressable locations exist. To address more memory, it is sometimes split in a set of banks which can first be selected.

An ATmega microcontroller (computer-on-a-chip) uses the Harvard-model (alternative for von Neumann) in which program memory and data memory are separate. It has two 16-bit address buses. It addresses memory in 64K banks.

ISA instructions locate operands in different ways:

- register: the operand is contained in a register, e.g. `mv X0,X1`
- immediate: the operand is contained in the instruction, e.g. `mv X0,#5`
- direct: the operand is stored at a specific memory address, e.g. `mv X0,(#31357718)`
- indirect: the operand is stored at a memory address pointed to by a register, e.g. `mv X0,(X1)`
- indexed: the operand is an array element the index of which is stored in a register (base and offset can be immediate or in register), e.g. `mv X0,(X1+X2)`, `mv X0,(X1+5)`, `mv X0,(#12345+X2)`
- stack: the operand is located on a stack, e.g. `mv X0,(X1++)`

All modes except immediate occur as source and destination. Often many modes are available for `mv` but fewer for ALU instructions.

13.3 Larger Structures

Two essential aggregate structures are

- An ***Array*** is a sequence of cells, each of which has the same type (bytes, words, pointers, ...). Cells are accessed by computing the address of the first cell plus the size of a cell times the index (in the sequence) of the cell of interest. This means access time to an element is constant and independent of the array size.

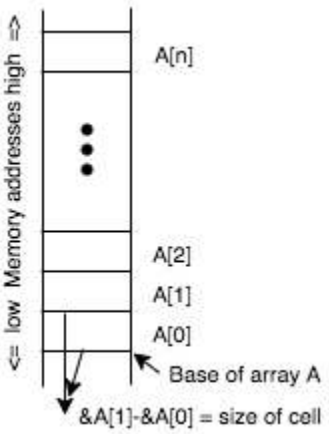


Figure 13.1: Array

- A ***Struct*** is a collection of cells each of which has a different type (but the types are known beforehand by the program that uses them). Cells are accessed by computing the address of the first cell plus the sum of the sizes of the cells in between. Note that a compiler 'knows' the layout of a struct and computes all offsets beforehand. This means access time to a field is constant and independent of the struct size.

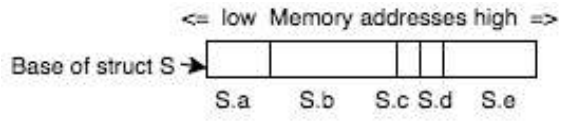


Figure 13.2: Struct

Arrays and structs are essential in all programming languages, though details may vary.

- A ***Stack*** is a structure which exhibits last-in-first-out behavior: the last element added is the first one taken out. At the ISA level it is implemented with an array and a pointer into that array. Adding an element means storing it and advancing the pointer. When the pointer reaches either end of the array, it's called stack over/underflow. Unless prevented by bounds checking this is an error situation (hence the website). Typically recursion and function calls use a stack.

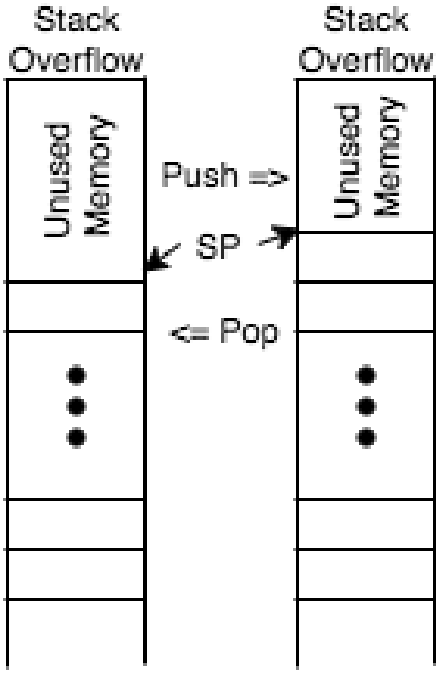


Figure 13.3: Stack

- A ***Queue*** is a structure which exhibits first-in-first-out (FIFO) behavior. It can be implemented using an array. Adding happens at one end just like in a stack, but removing happens at the other end. So there are two additional aspects:
  - A second pointer is used to keep track of the removal point. When the two pointers meet there is over- or underflow (depending on the operation)
  - When the insertion pointer meets the end of the array, it can be moved and insertions can continue (until the other pointer is met). The queue has two states: insertion above or below removal.

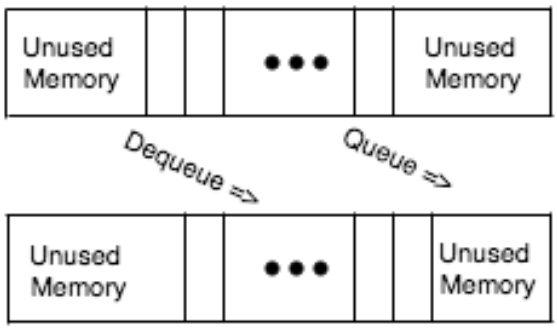


Figure 13.4: Queue

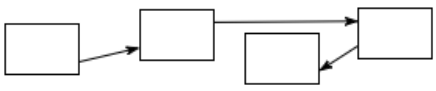


Figure 13.5: Linked List

- A ***Linked List*** is a set of structs each of which includes a pointer to another struct in the set (and other data). The order of structs in the list isn't determined by their address but is encoded in the links. A linked list can easily exhibit FIFO and LIFO behavior, so they are used for stacks and queues when performance isn't crucial.

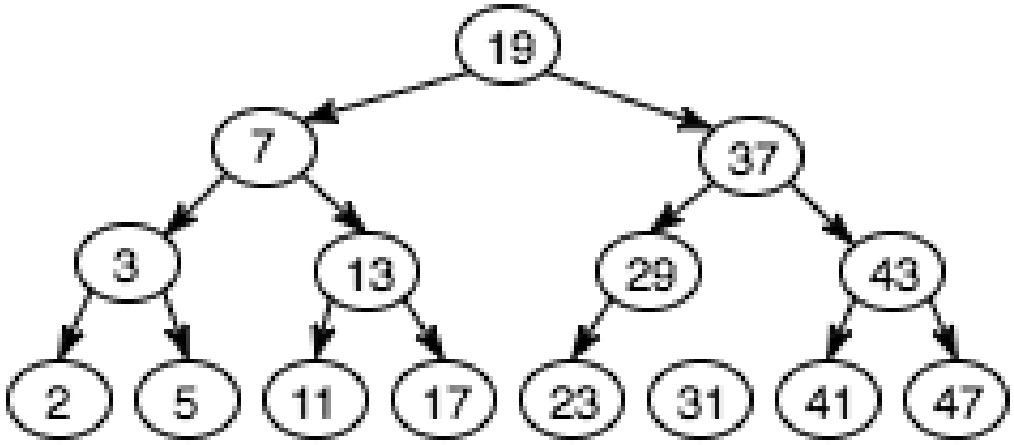


Figure 13.7: Binary Tree

- Many other pointer-based structures exist, optimized for different sorting and searching circumstances. For instance, using this ***binary search tree***, we can determine set-inclusion of a number by following at most three pointers.

13.4 Memory Pyramid

We've seen two types of memory: registers and DRAM. Registers are faster (closer to the ALU) but require more transistors (more chip surface). DRAM is slower (separate chip) but offers more bang-per-buck (lower cost per bit).

System Engineering is about finding the perfect balance between cost and performance. This leads to the Memory Pyramid, with height indicating speed, and width indicating size.

- At the top are ***registers***: incredibly fast but small in number because only so much can be placed next to the ALU. *100s of bytes, 1ns speed, $/byte*
- Then there are ***caches***: high-speed memory located in the CPU. *MBytes, 3ns, $/Kbyte*
- Then there is ***RAM***: still relatively fast, and much larger. *GBytes, 50ns, $/Mbyte*
- Local secondary storage, much slower but comparatively huge in size
  - Solid State Disk ***SSD***. *100s GBytes, 100s ns, $/Gbyte*
  - Hard Disk ***HD***. *TBytes, µs, $/Tbyte*
- Others (usually offline storage): tapes, laser disks, CD/DVD, etc.

13.4.1 Caches

A cache is a block of fast memory in which a copy of data from slower memory is kept in order to access that

data faster. When the cache's copy is altered, it is called *dirty* and it's up to the caching system to copy that data back to slow memory.

When a CPU fetches an instruction, the chunk of memory which includes that instruction (called a ***page***, typically 4K) is copied from memory to the cache. Since instructions are normally executed sequentially, the next instruction (or perhaps even the next several instructions) can then be read from the cache, saving time.

A Level 1 cache is located in a processor core (there can be more than one). Often there are two L1 caches in a core: one for data and one for instructions. A L2 cache is also located in each core, and holds both instructions and data. The L1 cache caches L2. Often there is also one L3 cache per CPU (i.e. shared by all cores).

Note that in a multi-core system L1 and L2 caches introduce an engineering challenge: a change by one core in its cache may invalidate another core's cache. If a second core loads a memory page stored in one core's cache, that page must be offloaded into an L3 cache.

13.4.2 Virtual Memory

The same principle is used to cache disk content in memory. This type of cache goes by the name Virtual Memory and will be discussed separately.

14. Language Processing

14.1 Assembler

A CPU fetches bits from memory and decodes and executes them, which means that bits are altered here and there. All a CPU sees are bits – numbers. People aren't too good at numbers. Programming a computer using bits or even hexadecimal would be disastrously complex.

The programming language of the ISA is called ***Assembly Language***, and the tool that translates Assembly to bits is called an ***assembler***.

Unlike other languages, assembly doesn't offer abstractions such as control structures (if, while) or data structures (arrays, structs).

Assembler offers

- the use of labels instead of addresses. A label can be used to jump (i.e. goto) a place or to store data
- the use of opcodes as a mnemonic for instructions
- pseudo-opcodes which do not lead to instructions but rather direct the assembler
- a somewhat mnemonic notation for addressing modes.

We could look into manuals and tutorials to study assembly, but there is a much more effective way: study it on your own computer. The ***GNU C compiler*** offers all one needs.

14.1.1 The GNU C compiler

Every Linux machine, VM or Mac either has `gcc` installed, or it's easy to do so. Gcc compiles C to executable, but can also produce assembly language code.

For the following example, we've compiled a small program. We've chosen weird numbers in order to recognize the resulting assembly code. This program is compiled using `gcc -S asmtst.c` (telling the Gnu C compiler to stop after producing an assembly code file), which produces the file `asmtst.s`. This was done on a Mac with an Intel processor.

```
1 #include <stdio.h>
2 int main() {
3     int s=55;
4     for (int i=20; i<=30; i++) {
```

```
5           s += i;
6       }
7       printf("%d\n",s);
8       return 0;
9   }
```

14.1.2 Assembly

* Labels are at the beginning of a line followed by a colon. They are used for jump-to (jmp) or jump-on-greater (jg). * Most lines have the form `opc oprnd` where `opc` is an opcode: the name of an ISA instruction, and `oprnd` are the zero or more operands that instruction requires. * *`%rbp`* is the frame pointer; local variables are indexed off that pointer * So `movl $55, -8(%rbp)` is a long move of the decimal value 55 into the local variable `s` which 'lives' 8 bytes below the frame pointer (*`%rbp`*) * A macro mechanism where a set of instructions can be given a name, such that using the name will lead to those instructions being inserted. * `cmpl $30, -12(%rbp)`
`jg LBB0_4` compares `i` (located 12 bytes below the frame pointer) with 30 and jumps (i.e. exits the loop) if it is greater. * Note that by using labels we avoid having to count the number of instruction bytes. The assembler translates the label to an appropriate offset. * The C language statement `s += i;` translates to three instructions: `i` is put in the general purpose register `eax`, then `s` is added to it, and then the result is stored in `s`.

```
Lcfi2:
    .cfi_def_cfa_register %rbp
    subq    $16, %rsp
    movl    $0, -4(%rbp)
    movl    $55, -8(%rbp)
    movl    $20, -12(%rbp)
LBB0_1:
    cmpl    $30, -12(%rbp)
    jg  LBB0_4
## BB#2:
    movl    -12(%rbp), %eax
    addl    -8(%rbp), %eax
    movl    %eax, -8(%rbp)
## BB#3:
    movl    -12(%rbp), %eax
    addl    $1, %eax
    movl    %eax, -12(%rbp)
    jmp LBB0_1
LBB0_4:
```

Figure 14.1: Assembly

14.1.3 Machine Language

Below is a snippet of the machine code taken from the executable of this program. The use of numbers such as 55, allows us to quickly determine the location of this snippet.

```
00000f30  55 48 89 e5 48 83 ec 10  c7 45 fc 00 00 00 00 c7  |UH..H....E......|
00000f40  45 f8 37 00 00 00 c7 45  f4 14 00 00 00 83 7d f4  |E.7....E......}.|
00000f50  1e 0f 8f 17 00 00 00 8b  45 f4 03 45 f8 89 45 f8  |........E..E..E.|
00000f60  8b 45 f4 83 c0 01 89 45  f4 e9 df ff ff ff 48 8d  |.E.....E......H.|
00000f70  3d 39 00 00 00 8b 75 f8  b0 00 e8 0d 00 00 00 31  |=9....u........1|
```

Figure 14.2: Machinecode

Using the knowledge we now have, we can already interpret this snippet to some degree.
Exercise:
if I tell you `c7 45` is `movl`, can you tell me if this machine is little endian or big endian?

The values at address `0f42...0f45` are `37 00 00 00`. `37` hex = 55 decimal, so the least significant byte occurs first, so this machine is little-endian.

14.1.4 Lower & Higher Level Languages

Assembly is called low-level because it doesn't offer any abstraction mechanisms beyond labels, opcodes and macros.

The programming language C, regarded by some as a generic assembler, at least offers named functions, control structures and data types.

Java, C#, JavaScript and Python offer Object Orientation, a complex scala of parameterized data types, name spaces, and much much more.

So, lower vs higher is a relevant distinction, but most languages are in fact high-level.

14.2 Functional vs Imperative Languages

All languages mentioned so far are **imperative**: statements (assignments) change an existing state.

One may wonder how a program could achieve anything without changing the state, but consider this Javascript statement:

```
console.log(55 + function f(x) { return x<=30 ? x+f(x+1) : 0; } (20));
```

Not a single assignment in sight. It just defines and calls a function. It does exactly the same as the C program shown earlier (feel free to test it in a browser).

A **functional language** is a language which isn't imperative. Javascript is both functional and imperative.

14.2.1 Is There Something Wrong With Imperative Languages?

Mutability is a source of bugs. Consider this snippet of code

```
1 X=<someDataStructure>; Y=X;
2 # This doesn't copy the data structure but rather Y points to
     the same structure
3 # Any change in the data structure Y also changes X. This can
     lead to very subtle bugs
```

In 1998 Ericsson announced the AXD301 telephone switch, containing over a million lines of Erlang, and reported to achieve a high availability of nine "9"s (source: wikipedia).
That's 99.9999999% uptime, less than 4 seconds per year of downtime!

One key reason: immutability. Erlang data structures can not be changed.

There is no other language in the world that is as reliable as Erlang (and derived languages such as Elixir).

14.3 Compilation

An assembler translates assembly into object code: a sequence of numbers. Such a process is called **compilation**. The C compiler compiles a program either into assembly or directly into object code. In general, compilation is the process of translating one (computer) language into another.

14.3.1 Source-to-ISA Compilation Phases

- Lexical Analysis. Combining character sequences into logical units called tokens: identifier, keyword, operator, etc.
- Syntax Analysis. Determining the grammatical structure of token-sequences into an 'abstract syntax tree' structure which reflects the containment hierarchy
- Semantic Analysis. Applying semantic aspects such as variable-scope.
- Intermediate Code Generation. Generate executable code for some abstract machine such as a VM (see below).

- Code Optimization. Apply heuristics (rules of thumb). For instance, mv X,(A) followed by mv (A),X stores a value and immediately fetches it. The second instruction is superfluous.
- Code Generation. Finally, generate actual ISA instructions.
- Symbol Table. All identifiers needed

14.3.2 Compilation, and what's wrong with it

The main disadvantage of compilation is portability. In order to work on a platform, the compiler must be implemented specifically for that platform (not only the ISA, of which there are a few hands full, but also the OS, of which there are many).

For C this needs to be done anyway: most OSs are written in C.

For other languages this development would be prohibitively expensive.

14.3.3 Compilation and Virtual Machines

The common solution to the problem of portability of compiled code is "Virtual Machines". This is a concept similar to using an ISA to provide a standard "virtual processor" which is implemented on different platforms by different microarchitectures.

Such a "Virtual Machine" will provide a standard target for a language, or a group of languages, somewhat similar to an ISA but also offering many 'higher level' aspects, such as object orientation, data structures, memory management and so on.

Now portability has been separated from language development; the higher level language compiler translates to the standard language for the virtual machine. This means that porting the language(s) to a different platform only requires implementing the Virtual Machine on that platform; once that is done, any and all languages which are made for that Virtual Machine will work on the new platform.

The language compiler itself is often written in its own language, so once the Virtual Machine has been implemented on a new platform, the compiler will then work on that platform as well.

The Virtual Machine is generally written in C and requires only mild adaptations to any new platform, so porting the VM is usually straightforward.

14.3.4 Some Common Virtual Machines

<small>Note that these virtual machines are entirely different from the VMs used to run, say, Linux on a Windows machine</small>

- **.Net CLR**: One of the many reasons why the .Net platform is successful is that it is built around a well-documented virtual machine which is used as the target for tens if not hundreds of languages including Java and C#.
- **JVM**: the Java Virtual Machine today is also well documented and supports many source languages
- **Beam**: the Erlang virtual machine, supports a handful of languages aimed at its specific conceptual model
- **Python Virtual Machine**: supports mostly Python and derivatives

14.4 Interpretation

Once a compiler has translated a source program into the 'machine language' of a virtual machine, there are two ways to proceed.

One way is further compilation: on most platforms, .Net "Common Language Runtime" (CLR) code is compiled down to ISA machine code. The beauty is that CLR code can be distributed and all platform specifics are dealt with below that level.

Another way is ***Interpretation***. Just as the μarchitecture fetches, decodes and executes ISA instructions, so can we write a program that fetches, decodes and executes, say, JVM instructions.

An interpreter is a program that interprets instructions in some language one by one, and mimics their effect on the machine state.

14.4.1 Bytecode

Few languages are interpreted at the source code level, because that would mean that the analysis of the source text (syntax, grammar, validation) would have to be done repeatedly. If the source language is high-level, any sensible interpreter would first analyze the source text and translate it to some intermediate language, suited for direct interpretation.

In practice, any such intermediate language is modeled as a virtual machine. Since the instructions of this machine are commonly represented as numbers, or bytes, (just as ISA instruction), the language for such a machine is often called bytecode.

14.4.2 Reverse Polish Notation (RPN)

How much is `(((5-3)*(7+1)-(6-4)*(4-3))+((5-2)*(2-5)+(6-3)*(4-2)))`?

How would one generate ISA-level code that computes this? One approach would be to assign different registers to each pair of parentheses: mv $5,Rx; sub Rx,3, and so on.

However, this is wasteful of registers and doesn't scale for more complex computations.

Reverse Polish Notation (RPN) is mathematics, but it plays an important role in compilers and abstract machines (and a few other areas).

In RPN, the operator is written **behind the arguments** instead of in between. Not `5*6` but `5 6 *`.

Suddenly, parentheses are no longer needed!

`(((5-3)*(7+1)-(6-4)*(4-3))+((5-2)*(2-5)+(6-3)*(4-2)))` becomes

`5 3 - 7 1 + * 6 4 - 4 3 - * - 5 2 - 2 5 - * 6 3 - 4 2 - * + +`

It's different, but is it better?

To compute RPN we need a stack (to hold the data, operands and intermediate results). To compute an expression: from left to right, each number is pushed onto the stack, and for each operator, the operands are popped off the stack and (very important) the result is pushed back onto the stack.

Here, below every item the content of the stack is shown **when that item is processed**

```
5  3  -  7  1  +  *  6  4  -  4  3  -  *  -  5  2  -  2  5  -  ...

   5  3  2  7  1  8  16 6  4  2  4  3  1  2 14  5  2  3  2  5 -3  ...
      5     2  7  2    16  6 16  2  4  2 16    14  5 14  3  2  3
               2          16    16  2 16          14    14  3 14
                                    16                   14
```

Figure 14.3: RPN

```
1 5  3  -  7  1  +  *  6  4  -  4  3  -  *  -  5  2  -  2  5  -
    ...
2
3   5  3  2  7  1  8  16 6  4  2  4  3  1  2 14  5  2  3  2  5
    -3  ...
4      5     2  7  2    16  6 16  2  4  2 16    14  5 14  3  2
     3
```

```
5                  2           16     16   2 16            14     14   3
   14
6                                      16                          14
```

RPN can handle arbitrarily complex expressions. It can be optimized by using registers to make it faster than simply using the memory-based stack. Interesting note: in the earlier phases, compilers often use RPN!

14.4.3 PDF

Some languages are built entirely around RPN. Today, one notable example is PDF. If you open the contents of a pdf document as a text-file, you'll see lots of encoded data, but also snippets written in the RPN-based PDF programming language.

14.5 Language Processing

Most modern language processing systems consist of something like the following phases:

```
1        compile                compile
2 source =======> intermediate =======> bytecode
3
4
5 bytecode: interpreted by VM
6
7   or
8           compile                compile
9 bytecode =======> C/assembler =======> ISA: interpreted by
    µArch
```

14.5.1 Linking

Libraries are an important tool in software development because they allow us to leverage earlier work. When compiling a program that uses libraries it wouldn't be sensible to have to compile the entire library again. But that poses a problem: when we call a function in the library, we must know its address. Where is that?

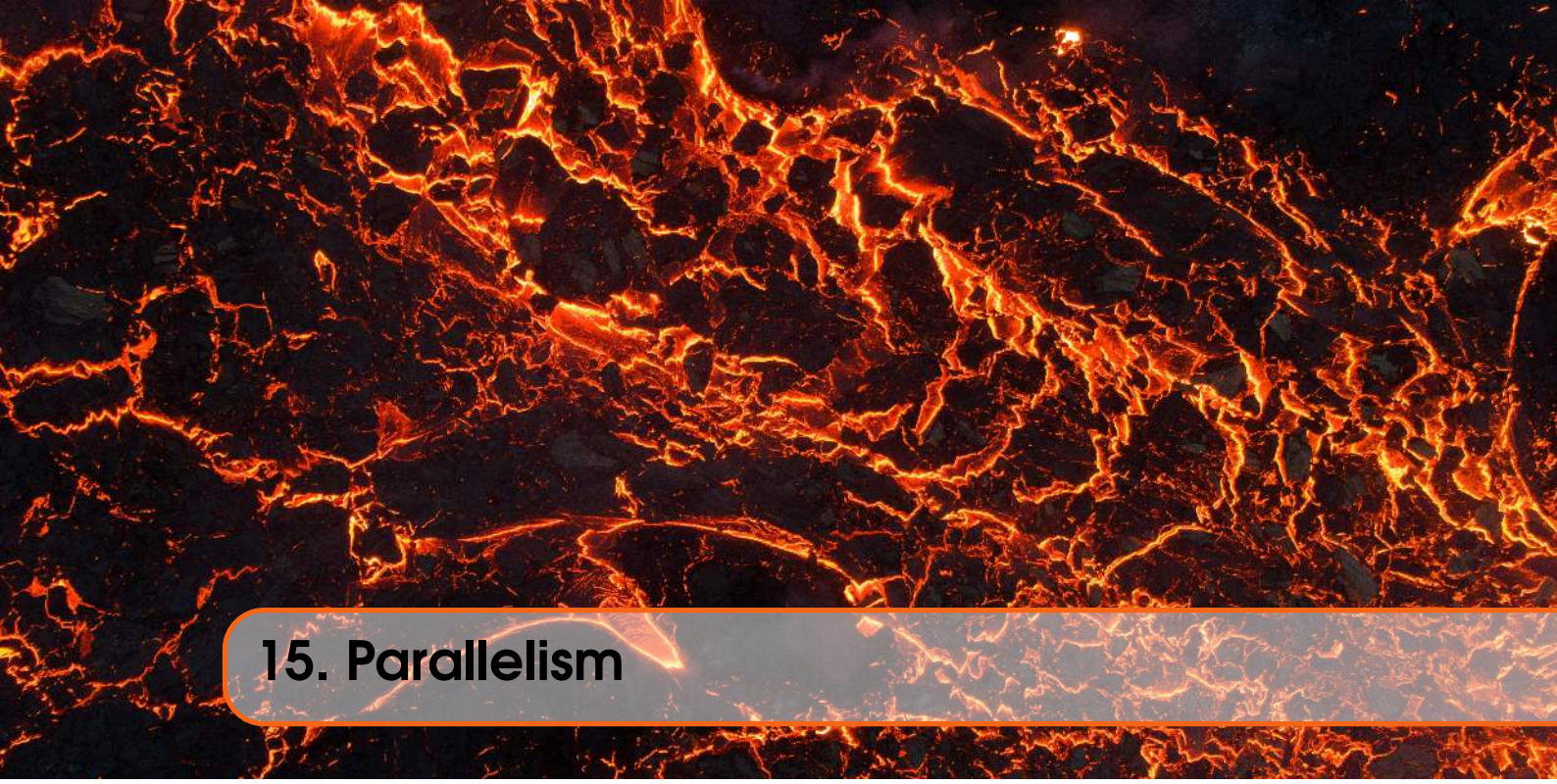
Running code using ***statically linked libraries*** requires an initial step: ***linking***. The unlinked code of the program and all libraries are loaded together with their symbol tables. Now the address of every module is known so if module A calls function F in module B, the reference in module A's symbol table can be replaced with the actual address of F in B.

Today, we also use ***dynamically linked libraries***. They are called using a slightly different mechanism which allows the OS to link them when the code is already running. There is a slight overhead.

14.5.2 JIT

Compilation takes time. For some languages, linked libraries are appropriate. However in some other circumstances, another principle is used: just-in-time compilation (JIT). The library isn't compiled, and maybe not even loaded until it is actually used. More likely, the libraries are compiled to bytecode, but the second phase (ISA-compilation and linking) happens "JIT".

Examples: .Net, Java, Python, V8 (JavaScript)



15. Parallelism

Users don't like to wait, and while a single ALU is fast, it can only do so much. System engineers have developed many ways to speed up processors. The most important idea is ***parallelism***. We've already seen some parallelism in the ALU. It computes several functions at the same time, then a multiplexer picks the result of interest.

Many different sections of a CPU can and do work in parallel. We'll only discuss a small number of aspects to give an overview.

15.1 Multiple Cores

Today, the largest desktop CPUs offer many cores: Intel i9: 18 cores; AMD Naples: 32 cores. For typical household use including gaming, about 16 cores is sufficient.

For servers, larger CPU's exist: Intel Xeon Phi: 72 cores. Ampere releases a 128-core ARM processor aimed at servers. Since designing chips becomes more and more automated and accessible, more and more companies (offering more and more jobs to System Engineers) are creating novel solutions.

Just adding cores is easy, but engineering the cache architecture and supporting hardware to keep many cores running effectively (using a single memory and network connection) is very difficult. In particular given the fact that **all software is aimed at the singular x86 or ARM model** (although software may be written to make use of multiple cores if they're there).

15.2 Pipeline

A core executes the fetch-decode-execute cycle. Naively, this would mean the fetch and decode circuitry is idle while the ALU is executing, but processors are somewhat predictable. When an instruction is executed, the next instruction is usually the one immediately following. That instruction can already be fetched and decoded while the first is executing.

Around 1980 the 5-stage pipeline was introduced, executing five aspects in parallel: instruction fetch, instruction decode, execute, memory access, register writeback.

However, the next instruction isn't always the one following. If the current instruction is a jump, the next instruction is at the place we are jumping to. This situation becomes known in the decode stage. Then, the fetch stage can be restarted to fetch the right instruction.

15.2.1 Conditional Branches

The situation becomes more complex for conditional branches, because the branch may or may not be taken *and the condition which decides this hasn't yet been computed*. Because any improvement is better than waiting, an entire field of engineering involves **branch prediction**.

Let's assume about 20% of instructions are conditional branches. An ideal 6-stage pipeline without prediction must therefore wait 20% of the time. This implies that instead of 1 cycle per instruction, it requires 0.8+0.2*6 cycles per instruction which is 2 cycles: the cpu runs at only half the best possible speed!

15.2.2 Branch prediction

Branch prediction heuristics include:<small>*(source https://danluu.com/branch-prediction/)*</small>

- *The branch will always be taken*. This is correct in perhaps 70% of the time. Already a big improvement
- *Backwards taken forward not*. Loops are executed more than once on average, so backward branches (in repeat..until) are usually taken, but forward branches (in if..., for... or while...) usually aren't because programmers tend to handle the likely case first
- *One bit*: maintain a table per branch with one bit indicating whether the branch was taken last time. This also works for branches based on locally stable conditions such as in sorting algorithms.
- *Two bit*: maintain a two-bit counter per branch. Works also for sequences of branches such as in a switch-statement.

Most of these heuristics were developed between 1990 and 2000. Current performance is in the area of 5% off 'always right' prediction!

15.3 Superscalar

Many instructions require more than one cycle to execute. Some common examples are floating point operations, and instructions that must fetch or store results in memory. During the execution of such an instruction, the pipeline and other hardware shouldn't be idle.

A **superscalar** is an extension on the pipeline, which allows multiple ALU functions to be executed at the same time. The ALU might for instance contain an additional integer arithmetic unit and a floating point unit. In that way, even though the instructions in isolation might take more than one cycle, a new operation can be started almost every cycle.

15.4 Simultaneous multithreading & Hyperthreading

Many processors advertise on the box: *n cores, m threads* (where m=2n). What's that about?

Every core has to wait less than a microsecond every now and then, when data or instructions aren't in a cache and must be fetched from memory. If the wait were longer, say for disk access, the OS could allow another process to run, but a process swap takes a few microseconds, so it isn't suitable in this case.

Simultaneous multithreading adds a second (or further) set of registers to a core. Intel's proprietary comparable solution (Hyperthreading) calls this a virtual core.

Whenever one thread needs to wait briefly, another set of registers is used to advance another thread. Obviously the pipeline and superscalar must be integrated for this to work.

15.4.1 Thread vs Process

We will look at processes and threads later on, but for the moment it is relevant to note that by definition threads always share memory, so simultaneous multithreading doesn't introduce global issues around shared resources.

To introduce 'hyper-processes', many issues around shared-resource contention and security would ensue. Far too complex to solve by hardware within microseconds.

But simultaneous multithreading doesn't introduce these issues. In desktop CPUs the maximum is 2 logical cores per real core, but server CPU's exist with more than that.

15.5 SIMD

Our processor model might be called SISD: a single instruction processes a few single data values. Consider that your favorite first-person shooter game generating 60 fps at, say, 1920x1080 resolution is performing roughly 120 million computations per second. Every second the same (or similar) instructions need to be fetched and decoded to perform the same calculations (on different data).

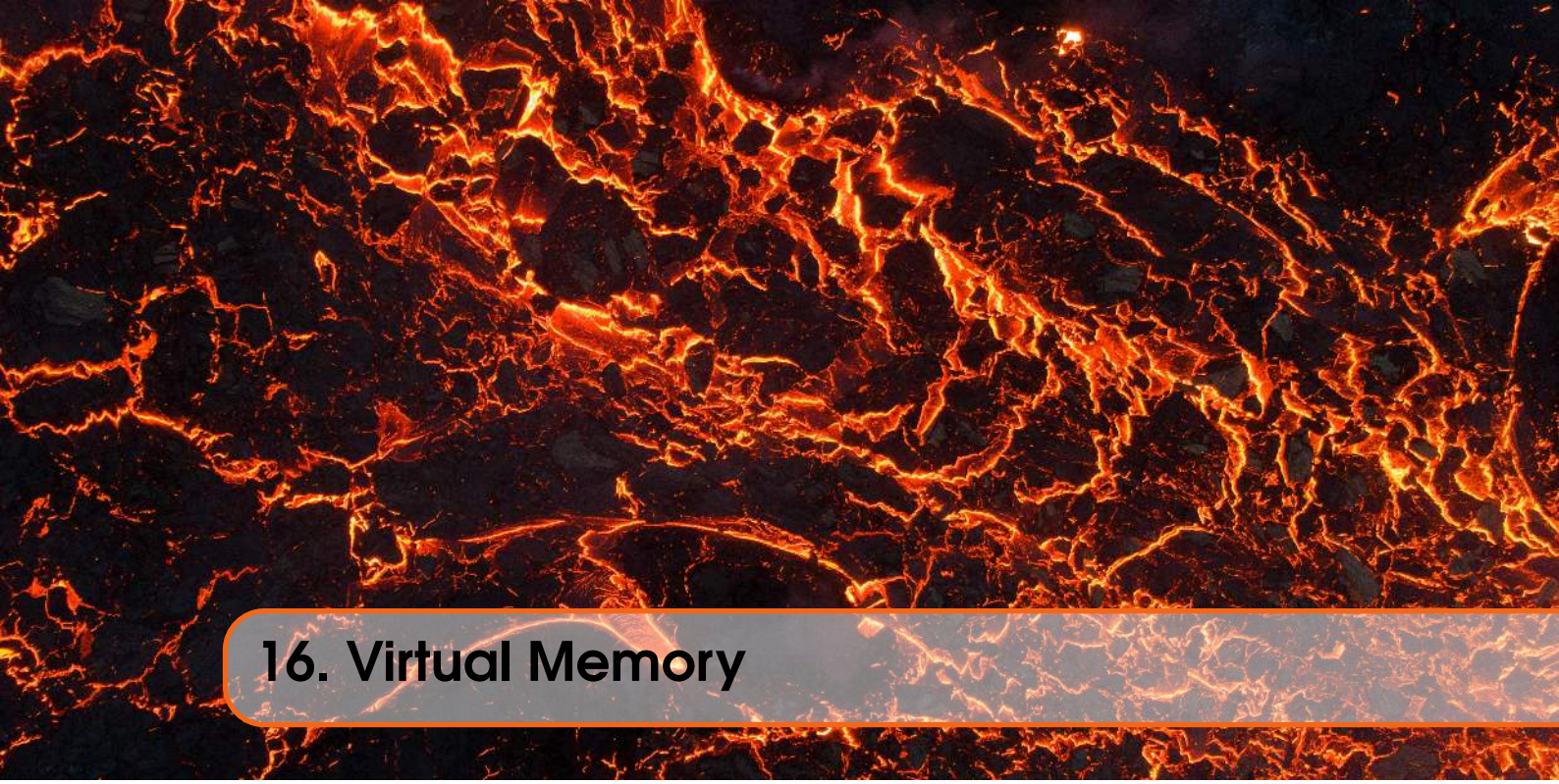
In an SIMD processor, a single pipeline drives multiple processing units (which implement some ALU functions). The array of processing units is often called the ***vector unit*** and SIMD is also called vector processing.

The key use-cases for this are graphics and crypto.

Today, the x86 has 32 512-bit vector registers supporting many floating point and integer SIMD operations.

Browser vendors were working on SIMD.js, platform-independent JavaScript access to vector instructions, but those activities have ceased in favor of:

WebAssembly: the ability to load and use assembler-like code in a browser. Supported on many different platforms. Yeah! `http://webassembly.org/demo/Tanks/`



16. Virtual Memory

In the early days of computing, memory was expensive and small. Programmers had to write small programs that used as little memory as possible. Larger programs were decomposed in **overlays**. For instance, each phase in a compiler wrote its results to a file, after which the next phase could be loaded (overlaying the previous) in memory.

As memory grew, the demands also grew, so programmers were spending significant time on this manual memory management. A model was engineered which solved this: **Virtual Memory**.

Conceptually, virtual memory means a program can use a lot more memory than is (physically) available. That virtual memory is simply stored on disk, and physical memory is used as a cache for the memory stored on disk (very much as a CPU cache temporarily stores memory content). Note that this is the same as a program storing intermediate results in a file, with one exception: the operating system must handle the virtual memory administration automatically.

16.1 Paging

The heart of virtual memory is **paging**. The entire memory a program uses is divided in **pages**. Whenever the program accesses a page which isn't currently stored in physical memory (this is called a **page fault**), that page is loaded into a section of memory of equal size, called a **frame**. When no unused frame is available (memory is full), the OS looks for a frame it can recycle, for instance a frame which hasn't been used for some time (the OS remembers last access times for all frames). If that frame is dirty it is first written back to disk. Then the page the program wanted to access is loaded into the now unused frame, and memory access can finally take place.

Alarm bells might ring: for **every** memory access we access a table, which is also stored in memory. Won't This Fail?

Good question. We already know part of the answer: page tables are accessed so frequently that they are usually cached.

The second part is that some specific hardware, called the **MMU** (Memory Management Unit), is responsible for the computations.

Thirdly, if the CPU (or rather a core) needs to wait a moment, a second logical core quickly

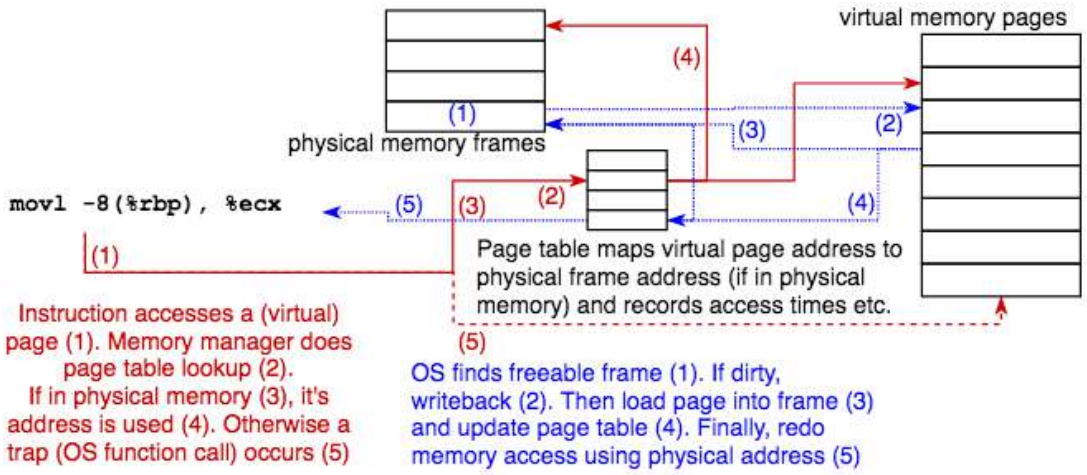


Figure 16.1: Virtual Memory

takes over to avoid waiting (multithreading).

16.1.1 Memory Management Unit, Translation Lookaside Buffer

An **MMU** is a dedicated piece of hardware (today embedded in the CPU) which controls all memory access (so it's located between the cores and the address bus), either before or after the caches.

Before looking into the Page Table, the MMU may look into an associative cache: a content-addressed lookup table called the **translation lookaside buffer** (TLB) which holds recently used pages. A TLB hit results in a physical address which can be used immediately. Only on a TLB miss will the page tables be traversed (there can be multiple levels).

The MMU generates a trap (i.e. invokes the OS) if a page isn't yet loaded in memory or if any error condition occurs =>

16.1.2 Page Table Entry

A page table entry (there's one per virtual memory page) holds the virtual and physical (if loaded) page addresses and various control information:

- a dirty bit
- an R/W bit (allows pages to be R/O)
- a bit indicating the required permission level (user or system)

If the virtual address points outside virtual memory, or if R/O or access rights are violated, the MMU generates a trap which invokes an error handler in the OS.

16.2 Segmentation

Segmentation stems from the time of overlays: instead of using a single contiguous memory space, a program divides its memory in separate segments, each holding code or data belonging to different aspects of the program. Segmentation is used for virtual memory, where segments are loaded and stored as the need arises, but it is also used as a programming structuring mechanism.

The main difference between paging and segmentation is that paging is automatic but segmentation is under program control. Note that segmentation and paging can be used together.

Segmentation is still being used today, and one of the most dreaded error messages when programming in C is 'segmentation fault, core dumped': indicating a memory access violation.

16.3 Page Size

There is no ideal page size: too small and the administration overhead becomes noticeable; too large and waste and performance loss might result.

The ARM MMU (included in all modern ARM processors) offers four sizes: 4k / 64k for two-level paging suitable for computers and phones (we'll discuss multi-level paging in a moment), or 1M / 16M for one-level paging more suitable for embedded multi-media systems.

The x86-64 MMU usually sticks to 4k pages, but does support larger 'hugepages'.

16.4 x86-64 Paging

Today, most Intel processors offer 4-level paging, but this architecture is limited to 256 TiB, so this Intel whitepaper `https://software.intel.com/sites/default/files/managed/2b/80/5-level_paging_white_paper.pdf` sketches the next gen 5-level architecture, good for 128 PiB.

This mode uses 57 bit addresses =>

<small>*Note: Whereas metric kilo, mega, giga, tera and peta count in powers of 1000, tera being 1000^4, the ISO and IEC promulgated standard counts in powers of 1024: kibi, mebi, gibi, tebi and pebi, a pebibyte being 1024^5 bytes, so 128PiB=1.4e17 B*</small>

16.5 x86-64 Five-level Paging

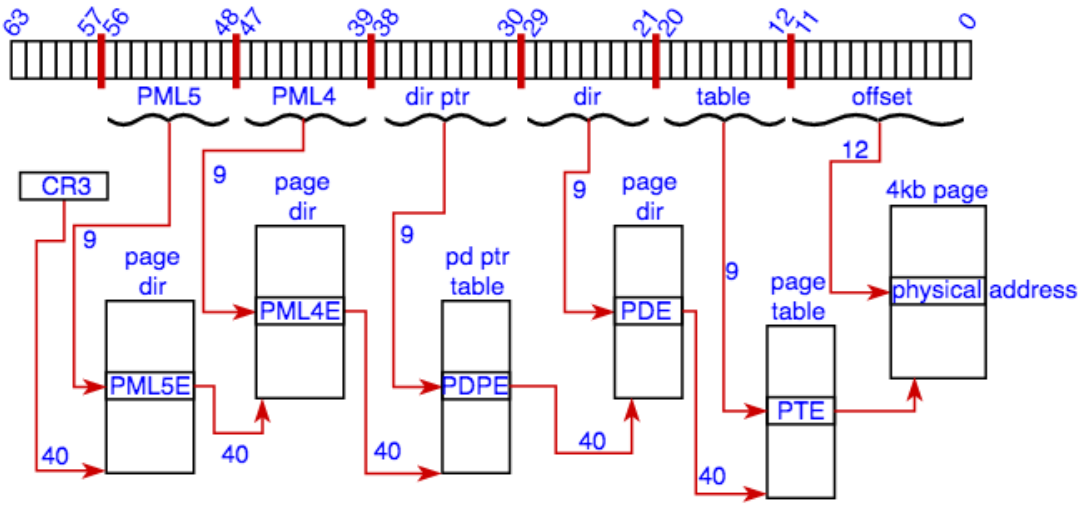


Figure 16.2: 5 Level Paging

- lower 12 bits are the offset in the physical page (2^12=4k)
- other layers use 9 bits = 512 entries (2^9=512) of 64 bits. A 40-bit base pointer and control various bits
- the base of the page-table tree is pointed to by register CR3
- each process has its own CR3 register which cannot simply be changed, thus every process is limited to its own memory
- in a virtualisation environment the higher levels are owned by the guest, but the lower levels are owned by the host. This way, guest memory management is as fast as host memory management

17. Multitasking

Although Multi-tasking, Concurrency and Parallelism all refer to the same concept, more than one thing happening at once, they are entirely different ideas.

Parallelism as we have discussed it, considers running a single, sequential ISA program, and aims to maximize execution speed by parallelizing activities to improve the speed of single program steps.

Concurrency considers multiple programs running at the same time on one system, possibly exchanging information. In English, the word concurrency primarily means 'at the same time'. In Dutch, and in Computer Science, the focus is on the fact that often only one program at a time can use some resource, so the parties are vying for that use!

Multi-tasking refers to the broad concept of computers seemingly doing many things at once. *Seemingly*, because actual parallelism isn't needed.

17.1 Multitasking

Early single-CPU computers allowed multiple users to use that CPU by giving every user in turn a (fixed) small amount of time on the CPU. The OS would allow user A's process to run for a tenth of a second, and then that process was interrupted and user B's process could run, and so on. When the only interaction is by keyboard, this kind of task switching is hardly noticeable.

Interrupting one process to allow another process to run is called **preempting** the first process. Obviously preemption isn't the only reason to suspend a process: when a process is waiting, for instance on disk access, it can also be suspended to allow another process to run.

The mechanism used to preempt is an **interrupt**. We already mentioned interrupts: the CPU quickly stores the PC and some relevant registers on the stack. The CPU state is elevated to get system rights, and the scheduler determines which process can now execute.

17.1.1 Concurrency issues

- running many programs in parallel, all using many different I/O devices with different speeds and properties, requires a complex *system of signals* being sent around. Subtle bugs (such as buffer overflow) may occur in rare circumstances

- mechanisms must ensure that only one program at a time may use certain resources (e.g. write to a file), but it is very difficult to ensure that *mutual exclusion* is correct in all circumstances
- when run in isolation the result of a program depends on the input, but if concurrent programs communicate or share a resource the result may depend on the way they are interleaved: *non-determinism*
- a program needs two resources. It obtains (exclusive rights to) the first and then waits until the second becomes available. Another program needs the same two resources and has obtained (exclusive rights to) the second and is waiting for the first resource to become available: *deadlock*

17.1.2 Non-Determinism

Ordinary programs are deterministic: given the same input they will do the same thing and produce the same output. Parallel programs sharing a resource such as memory can be non-deterministic. *(Note: despite the definitions given above, the terms 'parallel' and 'concurrent' are used interchangeably. Most people don't consider our technical interpretation of the term concurrent.)*

Consider for example two programs that share one variable `x`. Program P1 will just execute the statement `x = x + 3;`. Program P2 will just execute the statement `x = x * 4;`.

If `x` has the value 2 in the beginning, what is the value of `x` at the end?

```
1 x = 2;
2 P1: x = x + 3;
3 P2:‘x = x * 4;
```

If P1 runs first, 2 becomes 5, then P2 runs and 5 becomes 20

However, if P2 runs first, 2 becomes 8, then when P1 runs 8 becomes 11

So there are two possible outcomes? No wait: there are more

To compute `x = x + 3` P1 first fetches the value of `x`, then adds 3 and then stores the value in `x`. Likewise, P2 fetches `x`, multiplies by 4 and stores.

But what if P1 and P2 fetch the value of `x` at the same time?

P1 fetches 2 and stores 5, and P2 fetches 2 and stores 8. What is the value at the end?

It depends: who writes last? Possible outcomes are 5, 8, 11 and 20

And that is only true if reading/writing a value is one operation. What if values are 64 bits but there is only a 32 -bit data bus. The number of possible outcomes would be even bigger.

This is the wonderful world of **Non-determinism**, in this instance as a consequence of sharing a single variable.

Note: non-determinism isn't like a bug. Each of the possible outcomes is correct. *(This makes our earlier comment about a telephone exchange with 99.9999999% uptime written in Erlang, consisting of thousands of parallel processes all the more astounding)*

Writing correct concurrent software, which is deterministic or at least where the non-determinism doesn't harm the intended outcome, is **very difficult**.

There have been attempts to automate this, by writing compilers that take a sequential program and parallelize it. But the results aren't very good. At the moment, writing reliable concurrent software is still something only humans can do.

17.2 Mutual Exclusion

The problem in our P1|P2 example stems from the fact that both programs have unlimited access to the shared variable. What we want is **Mutual Exclusion**: while one program has access to `x`, the other doesn't.

17.2.1 Semaphores

A basic programming mechanism to offer this is a **Semaphore**: an OS-level mechanism which can limit access to a shared resource (such as variable `x`). Using a semaphore, the statement `x=x+3` can be made into a critical region. While one program has access the other will be suspended if it tries to access `x`.

At the heart a semaphore is made possible by an atomic Test-and-Set instruction: a hardware instruction which sets a flag and returns the value of the flag before it was set.

A program wishing to enter the critical region 'Test-and-Sets' the flag and then checks the result. Only if the return value indicates that no other program was 'using' `x` (the flag was not already set) will it proceed.

Semaphores allow us to manage non-determinism but not necessarily to prevent it entirely. Our earlier example still has two possible outcomes, depending on which program runs first.

And there is still room for deadlock

17.3 Deadlock

Suppose you are editing a huge photo of the Moon using the Gimp (an open-source Photoshop-like program). To do that, the Gimp needs a very large chunk of memory and it needs access to the photo file on your hard disk. Your OS swaps out as many processes as possible and is barely able to free the amount of memory needed.

So now the Gimp tries to access the file, but just at that moment your automatic disk compression program has started to compress this huge Moon photo file, **and has acquired unique access to it**. For such a large file the compressor needs more memory, and now it also asks the OS for a large chunk.

...

This is deadlock: process A has exclusive access right to resource a and needs access rights to resource b, but must wait on process B which holds exclusive access right to resource b but is waiting for access to resource a.

The primary task of an operating system (the kernel) is to manage resources - that is, to allow processes to make the most efficient use of resources.

Resources include:

- CPU time
- Memory
- Disk access (fine grained, e.g. per file)
- Network
- ...

Sometimes a resource can be shared (e.g. multiple processes reading one file), but sometimes access must be unique (e.g. writing to a file).

The problem with deadlock is that it is almost impossible to predict and not always easy to recognize. In our Moon-photo example, some other process might terminate, freeing its memory and thus resolving the deadlock.

In general, OSes try to detect deadlock by using many timers on all sorts of expectations. In the network-stack a timeout might simply indicate packet loss; in resource management it might indicate deadlock.

In some OSs deadlock might be prevented by requiring all programs to claim all resources beforehand. That way, when a program starts it is guaranteed to be able to finish.

Other OSs simply kill one program if they suspect it is part of a group of programs in deadlock.

18. Processes

A process is

- An executable program (i.e. ISA instructions)
- data used by that program (variables, stack, buffers, etcetera)
- an execution context = *current state*

The OS maintains administration of many aspects including:

- pending I/O
- resources sharing or exclusively using

18.1 Resource & Process Management

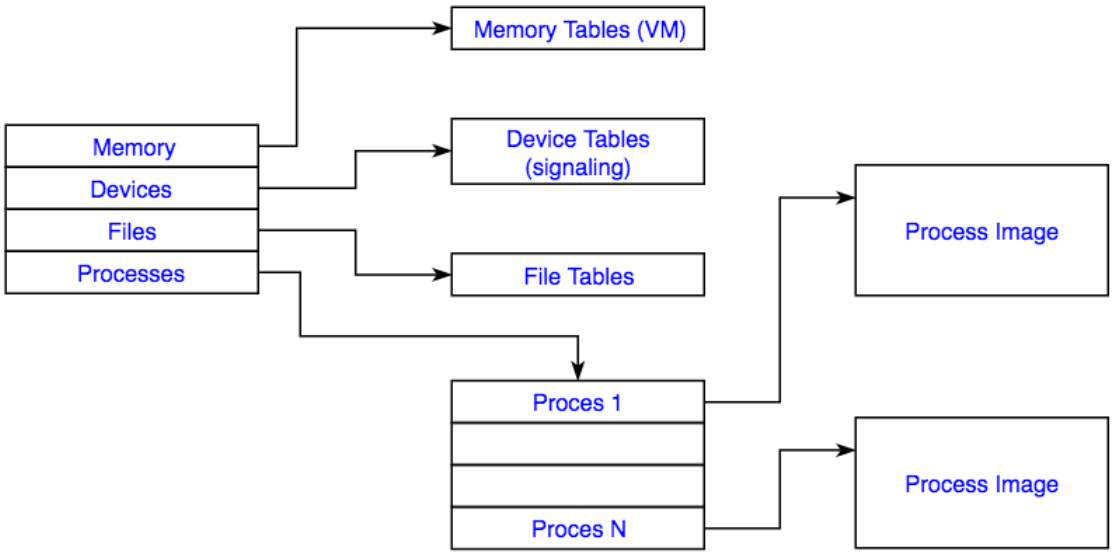


Figure 18.1: Process Table

- Memory tables (partly discussed for VM)
- I/O tables used to keep track of all ongoing I/O activity
- File tables contain information of the location and other properties of the hierarchical file system
- The process table contains information (state etc) on all known processes

18.1.1 Process Table

- User Program + Data (e.g. Stack)
- Process Control Block
  - ID's (this, parent, user)
  - Processor State
    - User-Visible Registers inc Stack Pointers
    - Control & Status (PC, Flags, Interrupt masks)
  - Process State
    - State, Prio, Scheduling Info, Blocking Event
  - Data structures e.g. all waiting
  - Interprocess Communications e.g. semaphores
  - Privileges e.g. root, quota
  - Memory Mngmnt
  - Resource ownership & Utilization (files, CPU use statistics)

18.1.2 Memory Management

OS responsibilities:

- **Process isolation**: programs should not be able to inspect or alter data from other programs (unless they are sharing data)
- **Access control protection**: mechanisms are in place to allow programs to share memory (in the entire hierarchy)
- **Automatic allocation and management**: Allocation in the hierarchy should be automatic and transparent to the programmer
- **Process relocation**: processes may need to be relocated (e.g. for swapping)
  - *Question: how is that possible?*
- **Modular programming support**: creation, destruction and change of program modules
- **Persistance**: access to long term storage

18.1.3 Information Protection

- **Availability**: an information service must be accessible for intended use (think of DDOS)
- **Confidentiality**: information must be accessible to intended parties and not to others (think of phishing)
- **Integrity**: information should only be altered by intended processes (e.g. XSS)
- **Authenticity**: information sources (when intended to be accessible) should be true (think of spoofing)
- **Accountability**: all changes can be traced to identities

18.2 Scheduling

which process gets access to the two main resources: CPU and memory?

- **Long term scheduling**: add to the pool of ready or suspended processes (allow into virtual memory)

- **Medium term scheduling**: add to the pool of ready or blocked processes from suspend (allow into memory)
- **Short term scheduling**: add to the pool of running processes (allow into CPU)
- **I/O**: management sleep/wakeup for I/O

18.2.1 Scheduling Goals

- **Fairness**: the current Linux scheduler is called *'Completely Fair Scheduler'*, which uses a **red-black-tree** (a *self-balancing binary search tree*) to maintain a timeline for every process adding real or virtual (when waiting) nanoseconds towards priority.
- **Differential Responsiveness**: individual applications may need priority (phone=>real-time scheduler, first-person shooter=> works better in windows)
- **Efficiency**: max throughput, min response time, min overhead

These goals are often conflicting

18.2.2 Short-term scheduling aka Dispatch

which ready process to execute next?

- invoked very frequently:
  - clock interrupts
  - I/O interrupts
  - OS calls (traps)
  - Signals (e.g. semaphores)
- criteria
  - priorities
  - responsiveness
  - throughput
  - qualitative criteria such as predictability

<small> *Source: Richard Stallings, Operating Systems: Internals and Design Principles* </small>

- first come first serve
- shortest process next
- shortest remaining time
- highest response ratio next (user info or past experience)
- feedback: give lower priority to long-running processes

18.2.3 Process State

Represents whether a process can run (or possibly why not). The book builds up to the seven state model, but we've added two here for run-level and preemption state.

- New: process has just been created
- Ready/Suspended: ready process is moved to disk to free up memory
- Ready: process is ready in memory waiting for CPU and not for I/O
- Running: process is currently active in the CPU
  - User state
  - System state
  - Preempted
- Blocked: process is waiting for I/O
- Blocked/Suspended: has been moved from memory to disk
- Zombie: finished, no resources but still in table (for parent)

	FCFS	Round Robin	SPN	SRT	HRRN	Feedback
Selection Function	max[w]	constant	min[s]	min[$s - e$]	max$\left(\frac{w+s}{s}\right)$	(see text)
Decision Mode	Non-preemptive	Preemptive (at time quantum)	Non-preemptive	Preemptive (at arrival)	Non-preemptive	Preemptive (at time quantum)
Throughput	Not emphasized	May be low if quantum is too small	High	High	High	Not emphasized
Response Time	May be high, especially if there is a large variance in process execution times	Provides good response time for short processes	Provides good response time for short processes	Provides good response time	Provides good response time	Not emphasized
Overhead	Minimum	Minimum	Can be high	Can be high	Can be high	Can be high
Effect on Processes	Penalizes short processes; penalizes I/O bound processes	Fair treatment	Penalizes long processes	Penalizes long processes	Good balance	May favor I/O bound processes
Starvation	No	No	Possible	Possible	No	Possible

Figure 18.2: Source: Richard Stallings, Operating Systems: Internals and Design Principles

18.2.4 Termination

- Normal completion
- Time limit exceeded
- No memory
- Bounds violation
- protection violation
- Arithmetic error
- Time overrun
- I/O error
- Invalid instruction
- Privileged instruction
- Data type error
- Intervention (e.g. deadlock)
- Parent terminated
- Parent request

18.3 Processes vs Threads

Process:

- 'Owns' resources
- Scheduling/Execution

But these are independent!

- Just execution: Thread

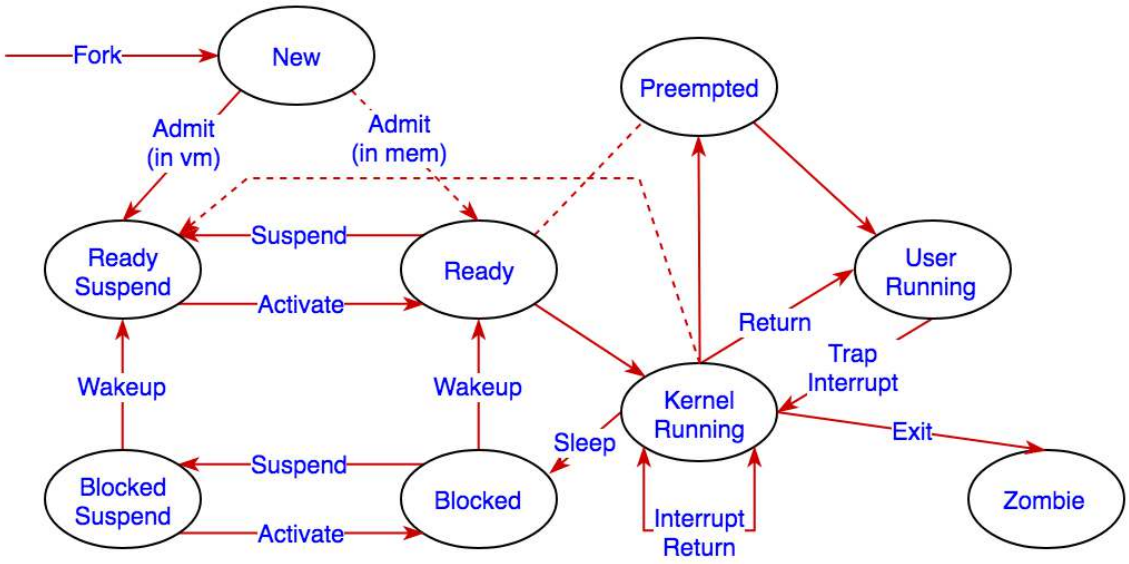


Figure 18.3: 9 State Transition Diagram

- Also resources: Process

Process has

- Virtual address space (& mem)
- Processor-time
- Access to other processes, files, I/O

Thread has

- State (PC, registers)
- Stack with local variables
- (Shared) access to process resources

Threads are 'light', Processes are 'heavy'

18.3.1 Thread Uses

- Foreground (UI) / Background (Logic, Data)
- Async
- Modularity
- Parallelism

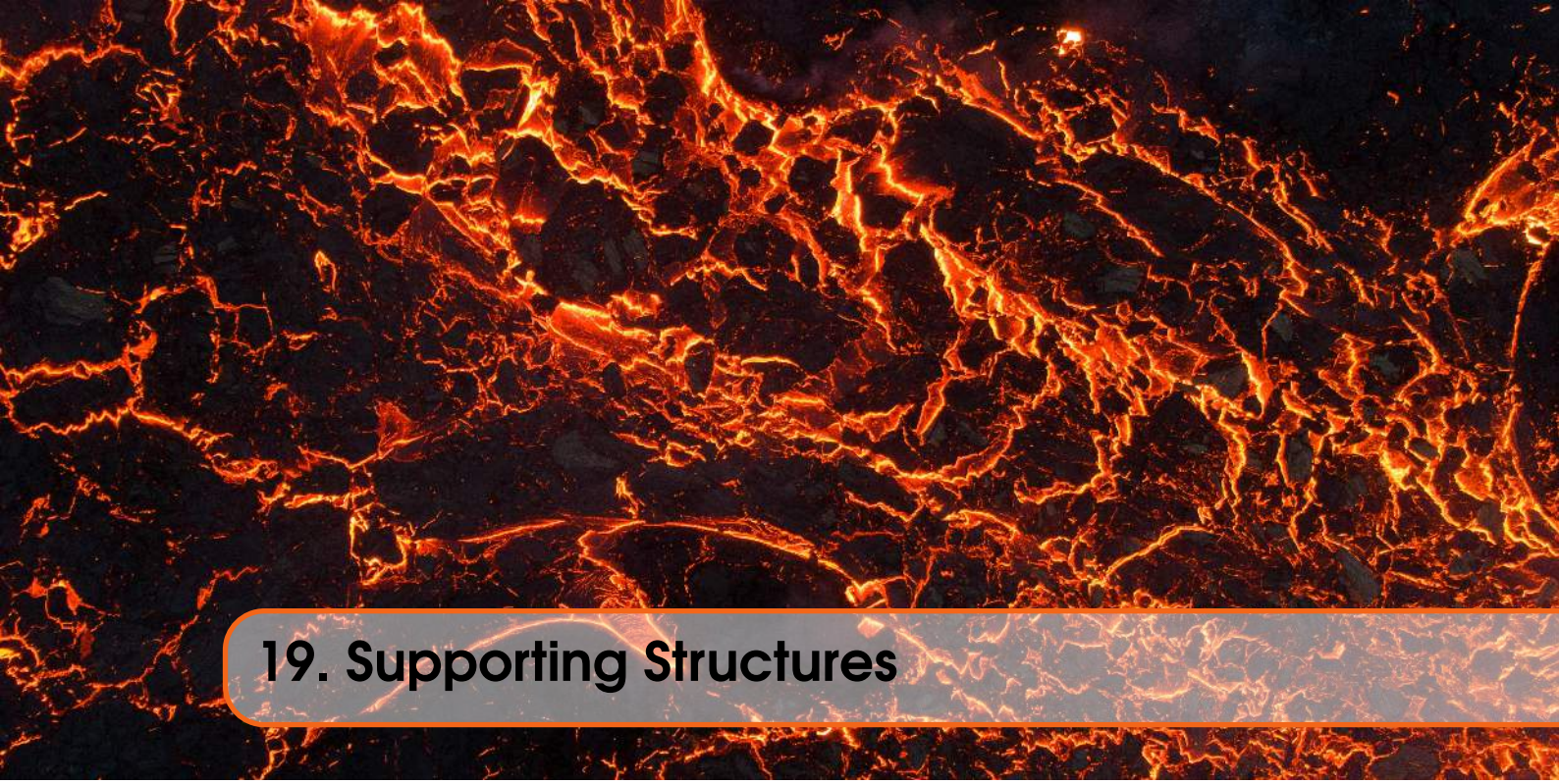
18.3.2 Thread Types

- User-level threads
  the application manages (using a thread library). The kernel doesn't 'see' threads (only the entire process has a state).

advantages: no mode switch, application specific scheduling, OS indep.
disadvantages: one blocks all, no multiprocessing,

- Kernel level threads: the kernel does management

main disadvantage: mode switch (can result in a factor 10 penalty)
Some OS's combine ULT & KLT



19. Supporting Structures

References

Special thanks to by friend Jamie for reviewing and sharing.
Photos from Pexels:
- Цвета Тишины
- Aaron Burden
- Aarti Vijay
- Anderson Guerra
- Andy Vu
- Björn Austmar Þórsson
- Fiona Art
- Francesco Ungaro
- Giovanni Figueroa
- Harrison Candlin
- Humphrey Muleba
- Isabella Mariana
- Jakob Rosen
- James Wheeler
- Kristina Chuprina
- Longxiang Qian
- Luis del Río
- Max Ravier
- Oday Hazeem
- Pixabay
- Pratik Gupta
- Roberto Nickson
- Rodrigo Souza
- Santiago Pagnotta
- Scott Webb
- Serkan Atay
- Sharon McCutcheon
- Silvia
- Simon Berger
- Sofía Rabassa
- Stefan Stefancik
- Steve Johnson
- Toa Heftiba Şinca
- Victor Dompablo
- Will Mu